Midterm Conceptual Check — Solutions

Lavender Jiang

March 5, 2026

Contents

1 Chapter 1: Vectors and Vector Spaces — Solutions **2**

2 Chapter 2: Random Variables and Probabilities — Solutions **7**

3 Chapter 3: Optimization and Calculus — Solutions **13**

4 Chapter 4: Embedding and Retrieval — Solutions **22**

5 Chapter 5: Model Ensembling and Generalization — Solutions **30**

6 Chapter 6: Dimensionality Reduction and Regularization — Solutions **37**

7 Chapter 7: Clustering — Solutions **46**

1 Chapter 1: Vectors and Vector Spaces — Solutions

Question 1. D.

The existence of a multiplicative identity is stated as: there exists a scalar $1 \in \mathbb{R}$ such that $\forall\, v \in V \colon 1 \cdot v = v$. This is indeed a vector space axiom, but option D incorrectly writes $\exists\, 1 \in V$, placing the identity inside the vector space rather than in the scalar field. As written, option D is **not** a correct property of a vector space.

Question 2. E.

All of A–D are valid inner product properties: positivity (A), commutativity (B), scalar linearity (C), and distributivity over addition (D). Since none of them is "not a property," the correct answer is E.

Question 3. A, B, E.

A. Valid. This is the standard dot product, which satisfies all four inner product properties.

B. Valid. For any symmetric positive definite matrix A, $v^\top A\, v'$ defines a valid inner product. The dot product is a special case where $A = I$. Since A is positive definite, $v^\top A\, v > 0$ for all $v \neq 0$, ensuring positivity. Symmetry of A ensures commutativity ($v^\top A\, u = u^\top A^\top v = u^\top A\, v$), and linearity follows from properties of matrix multiplication.

C. Not valid. Violates positivity: if $v = [0, 1]$, then $\langle v, v\rangle = 0$ even though $v \neq 0$.

D. Not valid. This is not even symmetric ($\langle v, v'\rangle \neq \langle v', v\rangle$ in general), violating commutativity. It also violates positivity, since the result can be negative.

E. Valid. Commutativity holds since $v(x)v'(x) = v'(x)v(x)$. Linearity and distributivity follow from linearity of integration. Positivity holds because $\langle v, v\rangle = \int_l^u v(x)^2\, \mathrm{d}x \geq 0$, with equality iff $v = 0$.

Question 4. D.

A d-dimensional vector space has exactly d basis vectors. You cannot have $d + 1$ linearly independent vectors in a d-dimensional space — any set of more than d vectors must be linearly dependent. Options A, B, C, and E are all true properties of a basis.

Question 5. A.

The inner product computes the scalar projection of one vector onto another, scaled by the length of the second vector: $\langle v, w\rangle = \|v\|\|w\| \cos\theta$. Diagram A

shows this projection. B shows the area of a parallelogram (related to the cross product magnitude). C shows vector addition. D shows the cross product, which produces a perpendicular vector.

Question 6. D.

$\text{span}(\{w_1, w_2\}) = \{c_1[1,0,0] + c_2[0,1,0] : c_1, c_2 \in \mathbb{R}\} = \{[c_1, c_2, 0]\}$. Every vector in this span has a third component of 0. The vector $[2,3,1]$ has a nonzero third component, so it is not in the span.

Question 7. A, C, E.

A. True. $c_1 w_1 + c_2 w_2 = [c_1, c_2, 0] = 0$ only when $c_1 = c_2 = 0$.

B. False. They are linearly independent (see A).

C. True. $\langle w_1, w_2 \rangle = 1 \cdot 0 + 0 \cdot 1 + 0 \cdot 0 = 0$.

D. False. $\mathbb{R}^3$ is 3-dimensional, so a basis needs 3 vectors. Two vectors can only span a 2-dimensional subspace.

E. True. w_1 and w_2 are linearly independent and span this subspace by definition.

Question 8.

(a) $\langle v, v' \rangle > 0$: the vectors point in similar directions (high similarity). $\langle v, v' \rangle = 0$: the vectors are orthogonal (no similarity). $\langle v, v' \rangle < 0$: the vectors point in opposite directions (dissimilar).

(b) Compute the inner product $\langle q, d_i \rangle$ for each document d_i. The document with the largest inner product is the most similar to the query, and thus the most relevant result.

(c) The basis vectors are $\{x^3, x^2, x, 1\}$ and the coefficients are $3, -2, 5, 1$. The Euclidean vector representation is $f(v) = [3, -2, 5, 1]$, since $v(x) = 3 \cdot x^3 + (-2) \cdot x^2 + 5 \cdot x + 1 \cdot 1$.

Question 9.

(a) An orthonormal basis is a set of vectors $\{w_1, \ldots, w_d\}$ that form a basis (linearly independent and spanning) where every pair is orthogonal ($\langle w_i, w_j \rangle = 0$ for $i \neq j$) and every vector has unit norm ($\|w_i\| = 1$).

(b) An orthogonal matrix is a square matrix B whose rows (or equivalently, columns) form an orthonormal set.

(c) Let the columns of B be $b_1, \ldots, b_d$. Since they are orthonormal, $\langle b_i, b_j \rangle = b_i^\top b_j = \delta_{ij}$ (1 if $i = j$, 0 otherwise). The (i,j) entry of $B^\top B$ is $b_i^\top b_j = \delta_{ij}$, so $B^\top B = I$. Multiplying both sides on the left by B and on the right by B^{-1} (which exists since $B^\top B = I$ implies $B^{-1} = B^\top$), we get $BB^\top = I$.

Back to Contents

Question 10 (PCA).

(a) Since w is a unit vector ($\|w\| = 1$), the orthogonal projection of x onto w simplifies to $P_w(x) = \langle x, w\rangle\, w$. This is the component of x in the direction of w, i.e., the vector in $\text{span}(\{w\})$ closest to x.

(b) The residual of the orthogonal projection is $r = x - P_w(x)$, i.e., the difference between the original vector and its projection. Orthogonal projection is desirable because it minimizes the distance (error) between x and its approximation in $\text{span}(\{w\})$: $P_w(x) = \arg\min_{v\in\text{span}(\{w\})} \|x - v\|$. The key property is that the residual r is orthogonal to w, meaning $\langle r, w\rangle = 0$.

(c) We should **minimize** the average residual norm and **maximize** the average projected vector norm. Minimizing the residual means we discard the least information when summarizing each x_i by its projection. Maximizing the projected norm means we capture the most variation in the data along w. These two criteria are equivalent because $\|x_i\|^2 = \|P_w(x_i)\|^2 + \|x_i - P_w(x_i)\|^2$, so minimizing the residual is the same as maximizing the projection (since $\sum \|x_i\|^2$ is constant).

(d) w_2 must be orthogonal to w_1 (i.e., $\langle w_1, w_2\rangle = 0$). We then apply the same criterion — maximize projected variance (or equivalently minimize residual) — but restricted to directions orthogonal to w_1. This ensures w_2 captures new information not already captured by w_1.

(e) The projected data is $AW \in \mathbb{R}^{n\times k}$. Each row i of AW is the vector of projections $[x_i^\top w_1, \ldots, x_i^\top w_k]$, giving the coordinates of x_i along the k directions. This reduces each data point from d dimensions to k dimensions.

(f) The projected variance is $\frac{1}{n}\|Aw\|^2 = \frac{1}{n} w^\top A^\top A\, w$. By the method of Lagrange multipliers (maximizing $w^\top A^\top A\, w$ subject to $w^\top w = 1$), we get $A^\top A\, w = \lambda w$. So w must be an eigenvector of $A^\top A$, and the variance equals λ/n. To maximize, we pick the eigenvector with the largest eigenvalue λ_1, giving w_1. For w_2, we repeat on the subspace orthogonal to w_1, which yields the eigenvector with the second largest eigenvalue λ_2, and so on. The iterative procedure in (c) and (d) is therefore equivalent to computing the eigen-decomposition of the covariance matrix $\frac{1}{n}A^\top A$, ordered by decreasing eigenvalue.

Question 11 (SVD).

(a) Since w_i is an eigenvector of $A^\top A$ with eigenvalue λ_i: $\|Aw_i\|^2 = w_i^\top A^\top A\, w_i = w_i^\top \lambda_i w_i = \lambda_i$. So $\|Aw_i\| = \sqrt{\lambda_i} = \sigma_i$. Factoring out the norm: $Aw_i = \|Aw_i\| \cdot \frac{Aw_i}{\|Aw_i\|} = \sigma_i v_i$, where $v_i = \frac{Aw_i}{\sigma_i}$ is Aw_i normalized to unit length.

(b) By construction $\|v_i\| = 1$. For $i \neq j$: $v_i^\top v_j = \frac{(Aw_i)^\top (Aw_j)}{\sigma_i\sigma_j} = \frac{w_i^\top A^\top A\, w_j}{\sigma_i\sigma_j} = \frac{\lambda_j\, w_i^\top w_j}{\sigma_i\sigma_j} = 0$, since the eigenvectors w_i, w_j are orthogonal ($w_i^\top w_j = 0$). Therefore the v_i are orthonormal.

(c) From part (a), column i of AW is $Aw_i = \sigma_i v_i$. Stacking these, $AW = V\Sigma$, where:

- $V \in \mathbb{R}^{n\times k}$ has orthonormal columns $v_1, \ldots, v_k$ (the left singular vectors of A). Each v_i represents the pattern of how the data points line up along direction w_i.
- $\Sigma \in \mathbb{R}^{k\times k}$ is a diagonal matrix with entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k > 0$ (the singular values of A). Each σ_i measures how spread out the data is along direction w_i.

(d) From part (c): $AW = V\Sigma$. Since W has orthonormal columns, $W^\top W = I$. Multiply both sides on the right by $W^\top$: $AWW^\top = V\Sigma W^\top$. Since $WW^\top = I$ (when $k = d$, i.e., we keep all directions), we get $A = V\Sigma W^\top$. This is the SVD of A.

(e) PCA is a problem: find the directions that maximize projected variance (or equivalently minimize reconstruction error) for centered data. SVD is a matrix factorization: any matrix A can be decomposed as $A = V\Sigma W^\top$. SVD is one way to solve PCA — the right singular vectors W give the principal components, the singular values σ_i give the standard deviations along each direction, and the left singular vectors V give the projected coordinates (up to scaling by Σ).

(f) B. An autoencoder with nonlinear activations learns a nonlinear mapping from data to a lower-dimensional representation (encoder) and back (decoder), generalizing PCA's linear projection. With linear activations and mean-squared error loss, an autoencoder recovers PCA. Options A and D are steps within linear PCA itself, and C is just an orthogonal transformation.

Question 12.

(a) The inner product $\langle q_i, k_j\rangle = \|q_i\|\|k_j\|\cos\theta$, where θ is the angle between the two vectors (Q5):

- **Large positive score:** q_i and k_j point in similar directions (θ small). Word j is highly relevant to word i — the model should pay strong attention to it.
- **Near zero:** q_i and k_j are roughly orthogonal ($\theta \approx 90°$). Word j is irrelevant to word i.
- **Large negative:** q_i and k_j point in opposite directions (θ near $180°$). Word j is actively dissimilar to what word i is looking for.

This is exactly the geometric interpretation from Q8(a): the inner product measures how aligned two vectors are.

(b) If entries of q_i and k_j are on the order of 1, then each term $q_{i,l} \cdot k_{j,l}$ in the sum $\langle q_i, k_j\rangle = \sum_{l=1}^{d} q_{i,l}\, k_{j,l}$ is on the order of 1. Summing d such terms gives an inner product on the order of d (or $\sqrt{d}$ in standard deviation if the terms are random with mean zero). As d grows, the raw inner products become very large in magnitude, which causes numerical issues when these scores are passed

through a softmax function (the exponentials explode or vanish). Dividing by $\sqrt{d}$ normalizes the scores so they remain on the order of 1 regardless of dimension, keeping the softmax well-behaved.

(c) If $\|q_i\| = \|k_j\| = c$ for all i, j, then:

$$\langle q_i, k_j \rangle = \|q_i\|\|k_j\| \cos\theta = c^2 \cos\theta = c^2 \cdot \cos(q_i, k_j).$$

Since c^2 is the same for all pairs, ranking by $\langle q_i, k_j \rangle$ is equivalent to ranking by $\cos(q_i, k_j)$. This tells us that the inner product is fundamentally measuring *directional alignment* — how similar two vectors are in terms of the angle between them. The magnitude only matters when norms differ across vectors (as in Q8, where popular posts had larger norms). When norms are equal, the inner product reduces to a scaled cosine similarity.

2 Chapter 2: Random Variables and Probabilities — Solutions

Question 1. E.

Both A and D are valid. Option A: $(0.3 + 0.2 + 0.1 + 0.3 + 0.1 = 1.0)$ with all entries non-negative. Option D: $(0.2 \times 5 = 1.0)$ with all entries non-negative (the uniform distribution). Option B sums to $1.1 \neq 1$, violating the sum-to-one axiom. Option C has a negative entry (-0.1), violating non-negativity.

Question 2.

(a) We compute $\exp(-e(a))$ for each activity:

$$\begin{aligned} &\exp(-1) \approx 0.3679, \quad \exp(-5) \approx 0.0067, \quad \exp(-3) \approx 0.0498, \\ &\exp(-2) \approx 0.1353, \quad \exp(-2) \approx 0.1353. \end{aligned}$$

The normalizing constant is $Z = 0.3679 + 0.0067 + 0.0498 + 0.1353 + 0.1353 \approx 0.6950$. Therefore:

$$\begin{aligned} &p(\text{nap}) \approx 0.529, \quad p(\text{zoomies}) \approx 0.010, \quad p(\text{knock}) \approx 0.072, \\ &p(\text{stare}) \approx 0.195, \quad p(\text{food}) \approx 0.195. \end{aligned}$$

The lowest-energy activity (nap, $e = 1$) gets the highest probability, confirming that lower energy = more likely.

(b) When all energies equal c, $\exp(-e(a)) = e^{-c}$ for all a. The normalizing constant is $Z = 5e^{-c}$, so $p(a) = e^{-c}/(5e^{-c}) = 1/5$ for all a. The distribution is **uniform** — equal energies mean no preference.

(c) As $e(a^*) \to -\infty$, $\exp(-e(a^*)) \to +\infty$, which dominates the normalizing constant. So $p(a^*) \to 1$ and $p(a) \to 0$ for all $a \neq a^*$. The distribution collapses to a point mass on a^*.

Question 3. B.

A probability vector $(p_1, \ldots, p_5)$ with $p_i \geq 0$ and $\sum_i p_i = 1$ lives on the 4-simplex Δ^4. It is $(K-1)$-dimensional for $K = 5$ outcomes because once 4 probabilities are fixed, the fifth is determined by $p_5 = 1 - p_1 - p_2 - p_3 - p_4$. Option A is wrong because Δ^5 would require 6 outcomes. Options C and D describe different geometric objects that do not enforce the probability axioms.

Question 4.

(a) Reading directly from the table: $P(c_1 = \text{nap}, c_2 = \text{zoomies}) = 0.10$.

(b) Using the definition of conditional probability:

$$P(c_2 = \text{zoom} \mid c_1 = \text{nap}) = \frac{P(c_1 = \text{nap}, c_2 = \text{zoom})}{P(c_1 = \text{nap})} = \frac{0.10}{0.30} = \frac{1}{3} \approx 0.333.$$

Back to Contents

(c) Summing each row across c_2:

$$\begin{aligned} P(c_1 = \text{nap}) &= 0.15 + 0.10 + 0.05 = 0.30, \\ P(c_1 = \text{zoom}) &= 0.05 + 0.15 + 0.10 = 0.30, \\ P(c_1 = \text{knock}) &= 0.10 + 0.05 + 0.25 = 0.40. \end{aligned}$$

Verification: $0.30 + 0.30 + 0.40 = 1.00$, so this is a valid probability distribution.

Question 5.

(a) Two valid chain rule factorizations:

$$\begin{aligned} &\text{Ordering 1:} \quad P(c_1, c_2, c_3) = P(c_1)\, P(c_2 \mid c_1)\, P(c_3 \mid c_1, c_2). \\ &\text{Ordering 2:} \quad P(c_1, c_2, c_3) = P(c_3)\, P(c_2 \mid c_3)\, P(c_1 \mid c_2, c_3). \end{aligned}$$

Any permutation of (c_1, c_2, c_3) gives a valid factorization.

(b) Ordering 1: $P(c_1)\, P(c_2 \mid c_1)\, P(c_3 \mid c_1, c_2)$. This matches the causal story because c_1 acts first (unconditionally), c_2 reacts to c_1, and c_3 reacts to both c_1 and c_2. Each conditional depends only on previously acting cats. This is the same idea behind autoregressive language models, where each token is predicted given all preceding tokens.

Question 6.

(a)

$$\begin{aligned} \mathbb{E}[f(a)] &= \sum_a p(a)\, f(a) \\ &= 0.3 \cdot 0 + 0.1 \cdot 2 + 0.15 \cdot 5 + 0.25 \cdot 0 + 0.2 \cdot 1 \\ &= 0 + 0.2 + 0.75 + 0 + 0.2 = 1.15. \end{aligned}$$

On average, 1.15 items are knocked off per day.

(b)

$$\begin{aligned} \mathbb{E}[f(a)^2] &= 0.3 \cdot 0^2 + 0.1 \cdot 2^2 + 0.15 \cdot 5^2 + 0.25 \cdot 0^2 + 0.2 \cdot 1^2 \\ &= 0 + 0.4 + 3.75 + 0 + 0.2 = 4.35. \\ \mathrm{Var}[f(a)] &= \mathbb{E}[f(a)^2] - (\mathbb{E}[f(a)])^2 = 4.35 - 1.15^2 = 4.35 - 1.3225 = 3.0275. \end{aligned}$$

The high variance reflects that most activities cause 0 items to fall, but "knock things off table" causes 5, creating high spread.

Question 7.

(a) The observed f values over 10 days are: $0, 0, 2, 5, 0, 1, 0, 0, 5, 1$. The Monte Carlo estimate is:

$$\hat{\mathbb{E}}[f(a)] = \frac{1}{10}(0 + 0 + 2 + 5 + 0 + 1 + 0 + 0 + 5 + 1) = \frac{14}{10} = 1.4.$$

Back to Contents

(b) Yes, 100 days would give a better estimate. By the **law of large numbers**, as $M \to \infty$, the Monte Carlo average $\frac{1}{M}\sum_m f(a^{(m)})$ converges to the true expectation $\mathbb{E}[f(a)]$. More samples reduce the variance of the estimate (the standard error scales as $1/\sqrt{M}$), so 100 samples would give roughly $\sqrt{10} \approx 3.2$ times smaller error than 10 samples.

Question 8.

(a) For $x \in \mathbb{R}^2$:

$$\log p(x;\mu,\Sigma) = \underbrace{-(x-\mu)^\top \Sigma^{-1}(x-\mu)}_{\text{(i) squared } \Sigma^{-1}\text{-norm of deviation}} - \underbrace{\frac{2}{2}\log 2\pi}_{\text{(ii) dimension constant}} - \underbrace{\frac{1}{2}\log\det\Sigma}_{\text{(iii) volume scaling}}.$$

Term (i) measures how far x is from the mean μ, using the geometry defined by Σ^{-1}. Terms (ii) and (iii) together form the normalization constant ensuring the density integrates to 1.

(b) The diagonal entries $\Sigma_{11} = 0.5$ and $\Sigma_{22} = 4.0$ are the variances of nap duration and purr volume, respectively. The off-diagonal entry $\Sigma_{12} = 0.3 > 0$ indicates **positive correlation**: longer naps tend to come with louder purring. The covariance matrix encodes both the spread along each dimension and the linear dependence between dimensions.

(c) This is exactly the squared norm induced by the inner product $\langle u, u'\rangle_{\Sigma^{-1}} = u^\top \Sigma^{-1} u'$ from Chapter 1, with $A = \Sigma^{-1}$ (symmetric positive definite). That is, $(x-\mu)^\top\Sigma^{-1}(x-\mu) = \|x-\mu\|^2_{\Sigma^{-1}}$. It generalizes the squared Euclidean distance by accounting for the covariance structure: each dimension's contribution is rescaled inversely by its variance, so deviations along high-variance directions are penalized less.

Question 9.

(a) Marginalizing over z:

$$\begin{aligned} p(\text{nap}) &= 0.5\cdot 0.7 + 0.3\cdot 0.1 + 0.2\cdot 0.2 = 0.35 + 0.03 + 0.04 = 0.42, \\ p(\text{zoom}) &= 0.5\cdot 0.1 + 0.3\cdot 0.6 + 0.2\cdot 0.2 = 0.05 + 0.18 + 0.04 = 0.27, \\ p(\text{knock}) &= 0.5\cdot 0.2 + 0.3\cdot 0.3 + 0.2\cdot 0.6 = 0.10 + 0.09 + 0.12 = 0.31. \end{aligned}$$

Verification: $0.42 + 0.27 + 0.31 = 1.00$.

(b) Each individual expert $p(a \mid z)$ has a strong peak (0.7, 0.6, or 0.6 on one activity). The marginal $p(a)$ is a **weighted average** of these experts, which blends their peaks into a smoother, more complex distribution $(0.42, 0.27, 0.31)$. This is the key idea behind *mixture of experts*: simple component distributions (experts) are combined via a latent variable to create a richer overall distribution. In general, mixtures can be multimodal even if each component is unimodal.

Back to Contents

Question 10.

(a) B. Cat B (uniform) is more unpredictable — you have no idea what it will do next. Cat A almost always naps, so it is highly predictable.

(b) For Cat A:

$$\begin{aligned} H(p_A) &= -\left[0.9\log_2 0.9 + 4\cdot 0.025\log_2 0.025\right] \\ &= -\left[0.9\cdot(-0.152) + 4\cdot 0.025\cdot(-5.322)\right] \\ &= -\left[-0.137 + (-0.532)\right] = 0.669 \text{ bits.} \end{aligned}$$

For Cat B (uniform over 5 outcomes):

$$H(p_B) = -5\cdot 0.2\log_2 0.2 = -\log_2 0.2 = \log_2 5 \approx 2.322 \text{ bits.}$$

Since $H(p_B) = 2.322 > H(p_A) = 0.669$, Cat B is indeed more unpredictable. The uniform distribution achieves the **maximum entropy** over K outcomes, equal to $\log_2 K$.

Question 11.

(a) Sort activities by decreasing probability and accumulate:

Activity	$p(a)$	Cumulative
nap	0.40	0.40
knock	0.25	0.65
food	0.20	0.85

At cumulative probability 0.85, we have $R_{0.85} = \{\text{nap, knock, food}\}$. The threshold is $c_q = 0.20$: every activity in $R_{0.85}$ has $p(a) \geq 0.20$, and every activity outside (stare = 0.10, zoomies = 0.05) has $p(a) < 0.20$.

(b) From the same sorted list: nap (0.40) + knock (0.25) = 0.65. So $R_{0.65} = \{\text{nap, knock}\}$ with threshold $c_q = 0.25$.

(c) The mode only tells us the single most likely activity (nap). But the HDR communicates a **credible set** that captures the cat's likely behaviors while preserving uncertainty. A prospective adopter learns that the cat will *probably* nap, knock things off tables, or demand food — not just that it naps the most. The HDR is the smallest such set (minimum "volume"), so it gives the tightest summary of the cat's behavior at a given confidence level. This avoids collapsing the distribution to a point estimate.

Question 12.

(a) We have $u = g(v) = v^2$, so $v = g^{-1}(u) = \sqrt{u}$ (taking the positive root since $v \in [0, 3]$). The derivative is:

$$\frac{dv}{du} = \frac{1}{2\sqrt{u}}.$$

Back to Contents

Applying the change of variables formula:

$$p_u(u) = p_v(\sqrt{u}) \cdot \left|\frac{dv}{du}\right| = \frac{1}{3} \cdot \frac{1}{2\sqrt{u}} = \frac{1}{6\sqrt{u}}.$$

The domain is $u \in [0, 9]$ (since $v \in [0, 3]$ and $u = v^2$).

(b) No, p_u is not uniform — it is proportional to $1/\sqrt{u}$, which is large near $u = 0$ and small near $u = 9$. Intuitively, the squaring function $u = v^2$ "stretches" the upper end of the v-range more than the lower end: the interval $v \in [0, 1]$ maps to $u \in [0, 1]$ (width 1), but $v \in [2, 3]$ maps to $u \in [4, 9]$ (width 5). So the same amount of probability mass gets spread over a wider range of u at larger values, making p_u smaller there.

(c) For $g(v) = [v_1^2,\ 2v_2]^\top$, the inverse is $g^{-1}(u) = [\sqrt{u_1},\ u_2/2]^\top$. The Jacobian of the inverse is:

$$\mathrm{Jac}_{g^{-1}}(u) = \begin{bmatrix} \frac{\partial v_1}{\partial u_1} & \frac{\partial v_1}{\partial u_2} \\ \frac{\partial v_2}{\partial u_1} & \frac{\partial v_2}{\partial u_2} \end{bmatrix} = \begin{bmatrix} \frac{1}{2\sqrt{u_1}} & 0 \\ 0 & \frac{1}{2} \end{bmatrix}.$$

The determinant is $\det = \frac{1}{2\sqrt{u_1}} \cdot \frac{1}{2} = \frac{1}{4\sqrt{u_1}}$. The determinant measures how much the transformation locally shrinks or stretches volume. When propagating probability from v-space to u-space, we must account for this distortion: $p_u(u) = p_v(g^{-1}(u)) \cdot |\det \mathrm{Jac}_{g^{-1}}(u)|$. Without the determinant correction, the resulting p_u would not integrate to 1.

Question 13.

(a) The change of variables formula gives us $p_u(u) = p_v(g^{-1}(u)) \cdot |\det \mathrm{Jac}_{g^{-1}}(u)|$. The two ingredients are: (1) the ability to evaluate p_v at $g^{-1}(u)$ — this is easy because $p_v = \mathcal{N}(0, I)$ has a known closed-form density, and (2) the ability to compute the determinant of the Jacobian of the inverse mapping g^{-1}, which accounts for how g locally stretches or compresses volume. Together, these let us compute the exact probability of any generated sample u without ever having to write down p_u explicitly.

(b) If $g = g_T \circ \cdots \circ g_1$, then $g^{-1} = g_1^{-1} \circ \cdots \circ g_T^{-1}$. By the chain rule for Jacobians, $\mathrm{Jac}_{g^{-1}} = \mathrm{Jac}_{g_1^{-1}} \cdots \mathrm{Jac}_{g_T^{-1}}$. Since $\det(AB) = \det(A)\det(B)$:

$$|\det \mathrm{Jac}_{g^{-1}}| = \prod_{t=1}^{T} |\det \mathrm{Jac}_{g_t^{-1}}|.$$

So the total volume distortion is the product of the distortions at each step. This is practical because each individual step g_t is designed to be simple (e.g., a small perturbation), making each $\det \mathrm{Jac}_{g_t^{-1}}$ easy to compute.

(c) Directly defining a complex distribution p_u over high-dimensional cat profiles is extremely difficult — we would need to capture all the correlations,

multimodality, and structure in the data, and ensure the density integrates to 1. Starting from $\mathcal{N}(0, I)$ is advantageous because: (1) it is trivial to sample from (just generate independent Gaussian noise), (2) it has a known, easy-to-evaluate density, and (3) all the complexity is offloaded to the learned transformation g, which can be trained from data. The change of variables formula guarantees that the result is a valid probability distribution as long as g is invertible and differentiable. This is the key insight behind normalizing flows and diffusion-based generative models.

3 Chapter 3: Optimization and Calculus — Solutions

Question 1.

Answer: B.

A. *False.* The empirical cost is a finite-sample average; it only equals the expected cost in the limit $N \to \infty$ (by the law of large numbers).

B. *True.* The training set $\{x_1, \ldots, x_N\}$ consists of i.i.d. draws from p_D, so $\frac{1}{N}\sum_i c(x_i, \theta)$ is a Monte Carlo estimate of $\mathbb{E}_{x\sim p_D}[c(x, \theta)]$.

C. *False.* Minimizing the empirical cost can overfit. The empirical minimizer may not coincide with the population minimizer, especially when N is small.

D. *False.* The per-instance cost can be any differentiable loss (MSE, cross-entropy, etc.). The 0-1 loss is not even differentiable.

Question 2.

(a) The full gradient averages all 6 per-instance gradients:

$$\nabla_{\theta_1} c(\theta) = \frac{1}{6}(2 + (-1) + 3 + 0 + (-2) + 4) = \frac{6}{6} = 1.$$

$$\nabla_{\theta_2} c(\theta) = \frac{1}{6}((-1) + 3 + 0 + (-2) + 4 + (-1)) = \frac{3}{6} = 0.5.$$

So $\nabla c(\theta) = [1,\ 0.5]$.

(b) The minibatch $\{x_2, x_4, x_6\}$ gives:

$$\tilde{\nabla}_{\theta_1} = \frac{1}{3}((-1) + 0 + 4) = 1, \quad \tilde{\nabla}_{\theta_2} = \frac{1}{3}(3 + (-2) + (-1)) = 0.$$

So $\tilde{\nabla} = [1,\ 0]$. This is *not* the same as the full gradient $[1,\ 0.5]$ — the θ_2 component differs. However, $\mathbb{E}[\tilde{\nabla}] = \nabla c(\theta)$: the stochastic gradient is an *unbiased* estimator. We prefer it because computing the full gradient requires processing all N data points, while the minibatch gradient uses only $B \ll N$ points, making each update much cheaper.

(c) Using the full gradient $\nabla c(\theta) = [1,\ 0.5]$:

$$\theta_1 = \theta_0 - \eta \nabla c(\theta_0) = \begin{bmatrix} 1.0 \\ 2.0 \end{bmatrix} - 0.1 \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.9 \\ 1.95 \end{bmatrix}.$$

Question 3.

(a) With $m_0 = 0$ and $\beta_m = 0.9$:

$$\begin{aligned} m_1 &= 0.9(0) + 0.1(4.0) = 0.4, \\ m_2 &= 0.9(0.4) + 0.1(-2.0) = 0.36 - 0.2 = 0.16, \\ m_3 &= 0.9(0.16) + 0.1(3.0) = 0.144 + 0.3 = 0.444. \end{aligned}$$

The momentum m_t is much smoother than the raw gradients (which swing from 4 to -2 to 3).

(b) With $v_0 = 0$ and $\beta_v = 0.99$:

$$\begin{aligned} v_1 &= 0.99(0) + 0.01(4.0^2) = 0.01(16) = 0.16, \\ v_2 &= 0.99(0.16) + 0.01((-2.0)^2) = 0.1584 + 0.04 = 0.1984, \\ v_3 &= 0.99(0.1984) + 0.01(3.0^2) = 0.196416 + 0.09 = 0.286416. \end{aligned}$$

(c)

$$u_3 = \frac{m_3}{\sqrt{v_3} + \epsilon} = \frac{0.444}{\sqrt{0.286416} + 0.001} = \frac{0.444}{0.5352 + 0.001} \approx \frac{0.444}{0.5362} \approx 0.828.$$

Dividing by $\sqrt{v_3}$ *normalizes* the update by the scale of recent gradients. If a parameter consistently has large gradients (large v_t), its effective step size is reduced; if it has small gradients, the step is amplified. This gives each parameter an adaptive learning rate, which is especially helpful when different parameters operate at very different scales.

Question 4.

(a) First compute the norm: $\|\tilde{\nabla}\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$. Since $5 > \tau = 2$, we clip:

$$\tilde{\nabla}_{\text{clipped}} = \tau \frac{\tilde{\nabla}}{\|\tilde{\nabla}\|} = 2 \cdot \frac{[3, 4]}{5} = \left[\frac{6}{5}, \frac{8}{5}\right] = [1.2,\ 1.6].$$

(b) Clipping *preserves the direction* of the gradient — it still points the same way in parameter space. It *changes the magnitude*, capping it at τ. This is useful because a single outlier minibatch (like the sea shanty binge) can produce an enormous gradient that sends parameters flying to a bad region of the loss landscape. Clipping limits the maximum step size, preventing such catastrophic updates while still moving in the correct direction.

Question 5.

(a) $c_{\text{MSE}} = (\hat{y}_i - y_i)^2 = (7.2 - 8.0)^2 = (-0.8)^2 = 0.64$.

(b) Under $y_i \sim \mathcal{N}(\hat{y}_i, \sigma^2)$ with $\sigma^2 = 1$:

$$p(y_i; \hat{y}_i, 1) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(y_i - \hat{y}_i)^2}{2}\right).$$

Back to Contents

Taking the negative log:

$$-\log p(y_i; \hat{y}_i, 1) = \frac{(y_i - \hat{y}_i)^2}{2} + \frac{1}{2}\log(2\pi).$$

The term $\frac{1}{2}\log(2\pi)$ is a constant (independent of θ), so minimizing the NLL is equivalent to minimizing $\frac{1}{2}(y_i - \hat{y}_i)^2$, which is proportional to the MSE. This shows that MSE implicitly assumes a Gaussian noise model: your predicted vibe score is the mean of a Normal distribution over possible ratings.

Question 6.

(a) Using the reference table:

$$e^{2.0} = 7.389, \quad e^{1.0} = 2.718, \quad e^{0.5} = 1.649, \quad e^{-1.0} = 0.368, \quad e^{0.1} = 1.105.$$

The partition function is $Z = 7.389 + 2.718 + 1.649 + 0.368 + 1.105 = 13.229$.

The softmax probabilities are:

$$\begin{aligned} p_{\text{pop}} &= 7.389/13.229 \approx 0.558, \\ p_{\text{indie}} &= 2.718/13.229 \approx 0.205, \\ p_{\text{hip-hop}} &= 1.649/13.229 \approx 0.125, \\ p_{\text{classical}} &= 0.368/13.229 \approx 0.028, \\ p_{\text{jazz}} &= 1.105/13.229 \approx 0.084. \end{aligned}$$

(b) The cross-entropy loss is:

$$c = -u_{y^*} + \log \sum_j \exp(u_j) = -1.0 + \log(13.229) = -1.0 + 2.583 = 1.583.$$

Equivalently, $c = -\log p_{y^*} = -\log(0.205) \approx 1.585$ (the small difference is rounding). Both expressions give the same result because $-\log p_{y^*} = -\log \frac{e^{u_{y^*}}}{Z} = -u_{y^*} + \log Z$.

(c) If $p_{\text{indie}} = 1.0$: $c = -\log(1) = 0$. The loss is zero when the model is perfectly confident and correct.

If $p_{\text{indie}} = 0$: $c = -\log(0) = +\infty$. The loss blows up, severely punishing a model that assigns zero probability to the correct class.

This makes cross-entropy a good differentiable proxy for 0-1 loss: it is minimized when the model assigns all probability mass to the correct class (matching 0-1 loss = 0), and it penalizes incorrect predictions increasingly as confidence in the wrong answer grows.

Question 7.

(a) Answer: B. Use the *validation set* for early stopping. The training loss always decreases (or we could just train longer), so it cannot tell us when to

stop. The test set must be held out until the very end. Only the validation set provides an unbiased signal for model selection.

(b) Using the test set to tune hyperparameters (like the learning rate) turns the test set into a second validation set. The reported "test performance" would then be optimistic — it reflects how well the model fits the test data, not how well it generalizes to truly unseen data. You lose the ability to make any trustworthy claim about real-world performance. The test set must be touched only once, at the very end, to get an honest performance estimate.

(c) This is **overfitting**. The model is memorizing training data (training loss decreases) but failing to generalize to new data (validation loss increases). The gap between training and validation cost is growing. You should **stop training at (or near) epoch 50** — the point where validation loss was lowest — and use the checkpoint saved at that epoch. This is early stopping.

Question 8.

(a) Each primitive:

- **Matrix-vector product** $Wa+b$: differentiable w.r.t. both a (input) and W, b (parameters). The Jacobian w.r.t. a is W; the gradient w.r.t. W is an outer product.
- $\tanh(\cdot)$: differentiable everywhere. Its derivative is $1 - \tanh^2(\cdot) \in (0, 1]$.
- **Dot product** $w^\top h + b'$: differentiable w.r.t. both h and w, b'.
- $\max(\cdot)$ (reduction): *not* differentiable at points where the maximum is achieved by more than one element (the derivative has a "kink"). However, these points form a set of measure zero, so in practice max is differentiable almost everywhere, and frameworks like PyTorch define subgradients at the ties.

(b) Without a nonlinearity, the composition of two linear maps is still linear:

$$W_2(W_1 a + b_1) + b_2 = (W_2 W_1)a + (W_2 b_1 + b_2) = W' a + b'.$$

This collapses to a single linear transformation — no matter how many layers you stack, the model can only represent linear functions. The pointwise nonlinearity (like tanh) breaks this linearity and enables the network to approximate nonlinear relationships (e.g., the interaction between tempo and energy that determines whether a user likes a song).

Question 9.

(a) Forward pass with $x = 1$, $w = 0.5$, $b = 2$, $y = 1.5$:

$$\begin{aligned} r &= wx = 0.5 \times 1 = 0.5, \\ h &= \tanh(r) = \tanh(0.5) \approx 0.462, \\ s &= bh = 2 \times 0.462 = 0.924, \\ d &= s - y = 0.924 - 1.5 = -0.576, \\ \ell &= d^2 = (-0.576)^2 \approx 0.332. \end{aligned}$$

(b) Backward pass — compute each factor:

$$\begin{aligned} \frac{\partial \ell}{\partial d} &= 2d = 2(-0.576) = -1.152, \\ \frac{\partial d}{\partial s} &= 1, \\ \frac{\partial s}{\partial h} &= b = 2, \\ \frac{\partial h}{\partial r} &= 1 - \tanh^2(r) = 1 - \tanh^2(0.5) \approx 0.786, \\ \frac{\partial r}{\partial w} &= x = 1. \end{aligned}$$

Multiplying:

$$\frac{\partial \ell}{\partial w} = \frac{\partial \ell}{\partial d} \cdot \frac{\partial d}{\partial s} \cdot \frac{\partial s}{\partial h} \cdot \frac{\partial h}{\partial r} \cdot \frac{\partial r}{\partial w} = (-1.152)(1)(2)(0.786)(1) = -1.811.$$

The negative sign means increasing w would *decrease* the loss (since the prediction $s = 0.924$ is below the target $y = 1.5$).

(c) To compute $\frac{\partial \ell}{\partial x}$, we use the same chain but replace the last factor:

$$\frac{\partial \ell}{\partial x} = \frac{\partial \ell}{\partial d} \cdot \frac{\partial d}{\partial s} \cdot \frac{\partial s}{\partial h} \cdot \frac{\partial h}{\partial r} \cdot \frac{\partial r}{\partial x}.$$

The first four factors are *exactly the same* as in part (b). Only the last factor changes: $\frac{\partial r}{\partial x} = w = 0.5$ instead of $x = 1$. This is the key insight of backpropagation: we compute the backward pass once from the loss to each node, and *reuse* the intermediate results. The shared prefix $\frac{\partial \ell}{\partial d} \cdot \frac{\partial d}{\partial s} \cdot \frac{\partial s}{\partial h} \cdot \frac{\partial h}{\partial r}$ only needs to be computed once, regardless of how many parameters or inputs we differentiate with respect to.

(d) The tape records each operation in order during the forward pass:

Step	`inputs`	`output`	`backward_fn`
1	$[x,\ w]$	r	`mul_backward`
2	$[r]$	h	`tanh_backward`
3	$[h,\ b]$	s	`mul_backward`
4	$[s,\ y]$	d	`sub_backward`
5	$[d]$	ℓ	`square_backward`

Back to Contents

(e) We iterate the tape in reverse, starting with seed $\bar{v}_\ell = 1$:

Step 5 (square): $\bar{v}_\ell = 1$. `square_backward`$(1, d) = 2d = 2(-0.576) = -1.152$. Accumulate: $\bar{v}_d \mathrel{+}= -1.152$.

Step 4 (sub): $\bar{v}_d = -1.152$. `sub_backward`$(-1.152, s, y) = (-1.152,\ +1.152)$. Accumulate: $\bar{v}_s \mathrel{+}= -1.152$, $\bar{v}_y \mathrel{+}= 1.152$.

Step 3 (mul): $\bar{v}_s = -1.152$. `mul_backward`$(-1.152, h, b) = (-1.152 \cdot b,\ -1.152 \cdot h) = (-2.304,\ -0.532)$. Accumulate: $\bar{v}_h \mathrel{+}= -2.304$, $\bar{v}_b \mathrel{+}= -0.532$.

Step 2 (tanh): $\bar{v}_h = -2.304$. `tanh_backward`$(-2.304, r) = -2.304 \cdot (1 - \tanh^2(r)) = -2.304 \cdot 0.786 = -1.811$. Accumulate: $\bar{v}_r \mathrel{+}= -1.811$.

Step 1 (mul): $\bar{v}_r = -1.811$. `mul_backward`$(-1.811, x, w) = (-1.811 \cdot w,\ -1.811 \cdot x) = (-0.906,\ -1.811)$. Accumulate: $\bar{v}_x \mathrel{+}= -0.906$, $\bar{v}_w \mathrel{+}= -1.811$.

Final result: $\bar{v}_w = -1.811$, which matches part (b). ✓

(f) If a variable is used as input to *multiple* operations, it receives a gradient contribution from each. For example, if h fed into both a multiply and an add, each backward step would produce a partial gradient for h. Using += correctly sums these contributions (by the multivariate chain rule: $\frac{\partial \ell}{\partial h} = \sum_k \frac{\partial \ell}{\partial o_k} \frac{\partial o_k}{\partial h}$ over all operations o_k that use h). Using = would overwrite the first contribution with the second, giving an incorrect gradient.

Question 10.

(a) When $m = 1$, the Jacobian J_ℓ is a $1 \times n$ row vector: $J_\ell = \begin{bmatrix} \frac{\partial \ell}{\partial x_1} & \cdots & \frac{\partial \ell}{\partial x_n} \end{bmatrix}$. The gradient $\nabla \ell$ is the *transpose* of this: $\nabla \ell = J_\ell^\top$. The gradient is a column vector (same shape as x), while the Jacobian is a row vector. They contain the same partial derivatives, just arranged differently.

(b) The Jacobian is:

$$J_f = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} x_2 & x_1 \\ 2x_1 & 1 \end{bmatrix}.$$

At $(x_1, x_2) = (3, 2)$:

$$J_f\big|_{(3,2)} = \begin{bmatrix} 2 & 3 \\ 6 & 1 \end{bmatrix}.$$

(c) The VJP is $v^\top J_f$ with $v = [1, -1]$:

$$v^\top J_f = \begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 6 & 1 \end{bmatrix} = \begin{bmatrix} 1 \cdot 2 + (-1) \cdot 6, & 1 \cdot 3 + (-1) \cdot 1 \end{bmatrix} = \begin{bmatrix} -4, & 2 \end{bmatrix}.$$

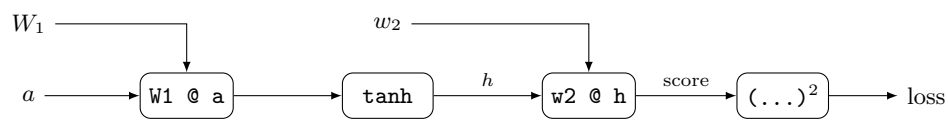
This is a single row vector (same dimension as the input), computed by one matrix-vector multiply.

(d) If $d = 10{,}000$, the full Jacobian is a $10{,}000 \times 10{,}000$ matrix with 10^8 entries — storing and computing this is prohibitively expensive. The VJP takes a d-vector in and produces a d-vector out in $O(d)$ time and memory, without ever

forming the full Jacobian. Since backpropagation only needs VJPs (one per node, propagated backward), it avoids the $O(d^2)$ cost entirely. This is why reverse-mode autodiff scales to networks with millions of parameters.

Question 11.

(a) The DAG is:



The leaves (inputs with `requires_grad=True`) are a, W_1, and w_2. The intermediate nodes are $W_1 a$ (matrix-vector product), $h = \tanh(W_1 a)$ (pointwise nonlinearity), score $= w_2^\top h$ (dot product), and loss $= (\text{score} - \text{target})^2$ (squared error).

(b) Let $s = w_2^\top h$ (score) and $\ell = (s-t)^2$ (loss). By the chain rule:

$$\frac{\partial \ell}{\partial w_2} = \frac{\partial \ell}{\partial s} \cdot \frac{\partial s}{\partial w_2} = 2(s-t) \cdot h.$$

Walking backward: $\frac{\partial \ell}{\partial s} = 2(s-t)$ (derivative of squaring), then $\frac{\partial s}{\partial w_2} = h$ (derivative of dot product $w_2^\top h$ w.r.t. w_2). The result is a vector the same shape as w_2.

(c) In the backward pass, we propagate a "gradient vector" $\bar{v}$ backward through each node by computing $\bar{v}^\top J_f$ (a VJP) rather than materializing the full Jacobian J_f. If an intermediate vector has dimension d, the full Jacobian is $d \times d$ (or larger), requiring $O(d^2)$ memory and computation. The VJP only needs $O(d)$: it takes a d-vector in and produces a d-vector out. In deep networks where d can be millions, this difference is critical.

(d) If we checkpoint every k layers, we store L/k checkpoint activations. During backward, to reconstruct a segment we recompute up to k layers from the nearest checkpoint, requiring k activations in memory. The total memory cost (in units of d) is:

$$M(k) = \frac{L}{k} + k.$$

This is a valid answer on its own. To find the optimal k, minimize:

$$\frac{dM}{dk} = -\frac{L}{k^2} + 1 = 0 \quad \Longrightarrow \quad k^* = \sqrt{L}.$$

Substituting back: $M(\sqrt{L}) = \sqrt{L} + \sqrt{L} = 2\sqrt{L}$, so the memory cost is $O(\sqrt{L} \cdot d)$.

The trade-off: memory drops from $O(Ld)$ to $O(\sqrt{L} \cdot d)$, but we pay extra computation — each segment of k layers is recomputed once during backward, totaling

up to one additional forward pass (∼33% overhead, since backward ≈ 2× forward). This is worthwhile for large models where memory is the bottleneck.

(e) Set `a = torch.tensor([2.0, 4.5], requires_grad=False)` (or simply omit `requires_grad`, since `False` is the default). The input a represents a fixed user profile — it is observed data, not a learnable parameter. We only want to update the model parameters (W_1, w_2), not the data itself. Computing gradients w.r.t. inputs is useful in other contexts (e.g., adversarial examples, input optimization), but not during standard training.

(f) The bugs are:

Bug 1: Wrong loss function. *Symptom: the loss decreases but the model produces nonsensical probabilities.* The code computes `-logits[..., batch_y].mean()`, which takes the negative of the raw logits for the correct class. This is missing the log-sum-exp normalization term. The correct cross-entropy loss (Eq. from lecture) is $-u^{y_i} + \log \sum_j \exp(u^j)$. **Fix:** Use `loss = nn.CrossEntropyLoss()(logits, batch_y)`, which computes the full cross-entropy including the log-partition function. *Relates to: cross-entropy loss (Q6).*

Bug 2: Missing `optimizer.zero_grad()`. *Symptom: the loss never converges and oscillates wildly.* Without zeroing gradients before `loss.backward()`, PyTorch *accumulates* gradients from every minibatch. After k minibatches, each parameter's `.grad` is the *sum* of k gradients rather than the gradient of the current minibatch. **Fix:** Add `optimizer.zero_grad()` before `loss.backward()`. *Relates to: SGD and minibatch gradients (Q2).*

Bug 3: Missing gradient clipping. While the order of `backward()` then `step()` is correct, there is no gradient clipping between them. If the loss landscape has steep regions, gradient magnitudes can explode and cause the parameters to diverge. **Fix:** Add `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` between `loss.backward()` and `optimizer.step()`. *Relates to: gradient clipping (Q4).*

Bug 4: Model selection on training loss instead of validation loss. *Symptom: accuracy on new users is much worse than the reported number.* The code picks the best model by evaluating on `train_loader`. This selects the checkpoint that overfits the most, not the one that generalizes best. **Fix:** Use a separate validation set: `val_loss = evaluate(model, val_loader)`. *Relates to: train/validation/test split and early stopping (Q7).*

Bug 5: Test evaluation on training data. *Symptom: the reported test performance seems suspiciously good.* The final performance report uses `train_loader` instead of a held-out test set. This gives an overly optimistic estimate of real-world performance. **Fix:** Use `test_loss = evaluate(best_model, test_loader)`. *Relates to: test set for reporting expected performance (Q7).*

Bonus Bug: Shallow copy of model. *Symptom: the "best" model somehow changes even after training ends.* `best_model = model` creates a reference, not a copy. As training continues, `best_model` and `model` point to the

same object, so the "best" model keeps changing. **Fix:** Use `best_model = copy.deepcopy(model)` (or save/load state dicts). *Relates to: checkpoint selection during early stopping (Q7).*

Back to Contents

4 Chapter 4: Embedding and Retrieval — Solutions

Question 1.

Answer: B.

A. *False.* Embedding functions are often nonlinear (e.g., deep neural networks). A linear transformation would be too restrictive to capture complex structure in images, text, or video.

B. *True.* Once posts are mapped to $\mathbb{R}^d$, we can use all the tools from linear algebra — inner products, distances, projections — to compare them, regardless of whether the original data was a photo, reel, or carousel.

C. *False.* For some simple data types, manual embeddings are possible, but for complex data like images or video, embedding functions are typically *learned* from data by training a model.

D. *False.* Embedding typically involves dimensionality reduction or abstraction, so some information is lost. The goal is to preserve *relevant* structure (e.g., semantic similarity), not all information.

Question 2.

Answer: B.

A. *False.* SVD finds the directions that *maximize* the variance of the projected data, not minimize it. Maximizing variance preserves the most information.

B. *True.* SVD finds the K orthogonal directions (right singular vectors) along which the data varies the most. Projecting onto these directions gives the best K-dimensional linear approximation of the data, which is useful for visualization.

C. *False.* SVD assumes the data lies near a linear (flat) subspace. If the data lies on a curved manifold (e.g., a swiss roll), SVD cannot recover the true low-dimensional structure.

D. *False.* SVD does not require unit-norm data. It works on the data as given (though centering is standard practice for PCA).

Question 3.

(a) SVD finds the best *linear* (flat, hyperplane) subspace to project onto. It assumes that the directions of greatest variance correspond to straight lines through the data. When the data lies on a curved manifold — like a style

spectrum that bends through the embedding space — projecting onto a flat subspace mixes together points that are far apart along the manifold. Two posts at opposite ends of the curve may project to the same point on the line, destroying the ordering. SVD cannot "unfold" the curve.

(b) No, $u+v$ is not guaranteed to lie on $\mathbb{S}$. If $\|u\| = 1$ and $\|v\| = 1$, then $\|u+v\|$ can range from 0 (when $v = -u$) to 2 (when $v = u$), but $\|u + v\| = 1$ only in special cases. For a concrete example: let $u = [1, 0]$ and $v = [0, 1]$, both on the unit circle. Then $u + v = [1, 1]$, and $\|u + v\| = \sqrt{2} \neq 1$. So $u + v \notin \mathbb{S}$. This is why the unit sphere is a *nonlinear* subspace — it is not closed under addition (or scalar multiplication).

Question 4.

(a) A and D are valid distance functions.

- **A.** *Valid.* The Euclidean distance $\|u - v\|$ satisfies all three properties: non-negativity (since norms are non-negative, with equality iff $u = v$), commutativity ($\|u - v\| = \|v - u\|$), and the triangle inequality.
- **B.** *Not valid.* The squared Euclidean distance violates the triangle inequality.
- **C.** *Not valid.* Cosine similarity is not even a distance function — it measures *similarity* (higher is more similar), not distance. It also violates non-negativity (it can be negative) and the triangle inequality.
- **D.** *Valid.* The Manhattan (L1) distance satisfies all three properties.

(b) For the squared Euclidean distance, here is a counterexample for the triangle inequality. Let $u = [0]$, $v = [2]$, $w = [1]$ in $\mathbb{R}^1$. Then:

$$\begin{aligned} d(u, v) &= \|u - v\|^2 = 4, \\ d(u, w) &= \|u - w\|^2 = 1, \\ d(w, v) &= \|w - v\|^2 = 1. \end{aligned}$$

We need $d(u, v) \leq d(u, w) + d(w, v)$, i.e., $4 \leq 1+1 = 2$. This fails, so the squared Euclidean distance is not a valid distance function.

For cosine similarity: it can be negative (e.g., $\cos([1], [-1]) = -1$), violating the non-negativity requirement $d(u, v) \geq 0$. It also equals 1 when $u = v$ rather than 0, so it does not satisfy $d(u, v) = 0 \iff u = v$.

Question 5.

(a)

$$Bu = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}, \qquad Bv = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}.$$

The transformed vectors are in $\mathbb{R}^2$ (reduced from $\mathbb{R}^3$).

Back to Contents

(b)

$$Bu - Bv = \begin{bmatrix} 2 \\ 3 \end{bmatrix} - \begin{bmatrix} 4 \\ 1 \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \end{bmatrix}.$$

$$\|Bu - Bv\|_I = \sqrt{(-2)^2 + 2^2} = \sqrt{4+4} = \sqrt{8} = 2\sqrt{2} \approx 2.83.$$

(c) This B projects from $\mathbb{R}^3$ to $\mathbb{R}^2$ by dropping the third dimension entirely and doubling the weight on the first dimension. The resulting distance "cares about" differences in the first two dimensions (with twice the emphasis on the first) and completely ignores the third dimension.

This is useful when the third dimension corresponds to noise, an irrelevant feature, or a dimension with high variance that does not reflect meaningful similarity. For instance, if the third embedding dimension captures post age or upload time, you may want to ignore it when measuring content similarity. Doubling the first dimension amplifies differences along that axis, which is useful if it captures the most semantically meaningful variation.

Question 6.

(a) With $q = [3, 1]$:

$$\begin{aligned}
\|v_1 - q\|^2 &= (4-3)^2 + (2-1)^2 = 1 + 1 = 2, \\
\|v_2 - q\|^2 &= (1-3)^2 + (0-1)^2 = 4 + 1 = 5, \\
\|v_3 - q\|^2 &= (3-3)^2 + (0-1)^2 = 0 + 1 = 1, \\
\|v_4 - q\|^2 &= (2-3)^2 + (3-1)^2 = 1 + 4 = 5, \\
\|v_5 - q\|^2 &= (5-3)^2 + (1-1)^2 = 4 + 0 = 4.
\end{aligned}$$

Ranking from most to least similar (smallest to largest distance): v_3 (1), v_1 (2), v_5 (4), v_2 (5), v_4 (5).

(b) Top-$K = 2$ retrieval, step by step:

- $D_0 = \{v_1, v_2, v_3, v_4, v_5\}$ is the full candidate set — all five posts we can recommend.
- $r_1 = \arg\min_{v \in D_0} \|v - q\|^2 = v_3$ (distance = 1). In words: scan every post in D_0, compute its squared distance to the query q, and pick the closest one. That is v_3, so $r_1 = v_3$ is our **first nearest neighbor**.
- $D_1 = D_0 \setminus \{r_1\} = \{v_1, v_2, v_4, v_5\}$. Remove the post we already selected (v_3) from the candidate set so we do not pick it again.
- $r_2 = \arg\min_{v \in D_1} \|v - q\|^2 = v_1$ (distance = 2). Repeat the search over the remaining candidates D_1. The closest is v_1, so $r_2 = v_1$ is our **second nearest neighbor**.

The top-2 nearest neighbors are $\{r_1, r_2\} = \{v_3, v_1\}$. In general, to retrieve top-K we repeat this process K times: find the closest remaining candidate, add it to the result set, and remove it from the candidate pool.

(c) No, the ranking would not change. The square root is a monotonically increasing function: if $a < b$, then $\sqrt{a} < \sqrt{b}$. So $\|v_i - q\|^2 < \|v_j - q\|^2$ implies $\|v_i - q\| < \|v_j - q\|$. The ordering is preserved, and the squared version avoids the computational cost of the square root.

Question 7.

(a) The term $-2\langle v, q\rangle$ captures the *relationship* between v and q. It is the only term that involves both vectors. The terms $\langle v, v\rangle = \|v\|^2$ and $\langle q, q\rangle = \|q\|^2$ depend only on the individual magnitudes of v and q, respectively.

(b) The term $\langle q, q\rangle = \|q\|^2$ is the same for all candidates (since the query q is fixed), so it can be ignored when ranking. That is, ranking by $\|v_i - q\|^2$ is the same as ranking by $\|v_i\|^2 - 2\langle v_i, q\rangle$. This saves one inner product computation per candidate. Note that the $\|v_i\|^2$ term *cannot* be ignored in general, because different candidates have different norms.

(c) If $\|v_i\| = c$ for all i, then $\|v_i\|^2 = c^2$ is the same for every candidate. Combined with the fact that $\|q\|^2$ is also constant, ranking by $\|v_i - q\|^2$ reduces to ranking by $-2\langle v_i, q\rangle$, which (after dropping the constant factor -2) is equivalent to ranking by $\langle v_i, q\rangle$ in *descending* order. A *larger* inner product means the candidate is *closer* (more similar) to the query.

Question 8.

(a) With $q = [1, 2]$:

$$\|v_1 - q\| = \|[4, 3] - [1, 2]\| = \|[3, 1]\| = \sqrt{9 + 1} = \sqrt{10} \approx 3.16,$$
$$\|v_2 - q\| = \|[2, 1] - [1, 2]\| = \|[1, -1]\| = \sqrt{1 + 1} = \sqrt{2} \approx 1.41.$$

By Euclidean distance, v_2 (the niche post) appears closer to the query.

(b) First compute norms and inner products:

$$\|q\| = \sqrt{1 + 4} = \sqrt{5},$$
$$\|v_1\| = \sqrt{16 + 9} = 5,$$
$$\|v_2\| = \sqrt{4 + 1} = \sqrt{5}.$$

Inner products:

$$\langle v_1, q\rangle = 4 \cdot 1 + 3 \cdot 2 = 10,$$
$$\langle v_2, q\rangle = 2 \cdot 1 + 1 \cdot 2 = 4.$$

Back to Contents

Cosine similarities:

$$\cos(v_1, q) = \frac{10}{5 \cdot \sqrt{5}} = \frac{10}{5\sqrt{5}} = \frac{2}{\sqrt{5}} \approx 0.894,$$
$$\cos(v_2, q) = \frac{4}{\sqrt{5} \cdot \sqrt{5}} = \frac{4}{5} = 0.800.$$

By cosine similarity, v_1 (the viral post) is more similar to the query.

(c) The two measures disagree because Euclidean distance is sensitive to vector magnitudes, while cosine similarity is not. The viral post $v_1 = [4, 3]$ has a large norm ($\|v_1\| = 5$), so it is geometrically far from the query in Euclidean space, even though it points in nearly the same direction. The niche post $v_2 = [2, 1]$ is closer in Euclidean space simply because it is shorter, not because its *direction* is more similar.

For a system that should surface relevant content regardless of popularity, **cosine similarity** is the better choice. It compares the *angle* (direction) between vectors, ignoring magnitude. Since magnitude tends to reflect training frequency (popularity) rather than semantic content, cosine similarity focuses on what the post is *about*, not how viral it is.

Question 9.

Answers: B, D, and E.

A. *False.* Cosine similarity does not satisfy the triangle inequality. For example, with $u = [1, 0]$, $v = [-1, 0]$, $w = [0, 1]$: $\cos(u, w) = 0$, $\cos(u, v) = -1$, $\cos(v, w) = 0$, but cosine similarity is a similarity (not a distance), and the triangle inequality does not hold in general.

B. *True.* $\cos(\alpha v, q) = \frac{\langle \alpha v, q\rangle}{\|\alpha v\|\|q\|} = \frac{\alpha\langle v, q\rangle}{\alpha\|v\|\|q\|} = \frac{\langle v, q\rangle}{\|v\|\|q\|} = \cos(v, q)$ for $\alpha > 0$. Scaling preserves direction.

C. *False.* An inner product must satisfy linearity: $\langle cv, q\rangle = c\langle v, q\rangle$. But $\cos(cv, q) = \cos(v, q)$ for $c > 0$, so scaling one vector does not scale the output. Cosine similarity is not linear, hence not an inner product.

D. *True.* If $\|v\| = \|q\| = 1$, then $\cos(v, q) = \frac{\langle v, q\rangle}{1 \cdot 1} = \langle v, q\rangle$.

E. *True.* $\cos(v, q) = 1$ means $\frac{\langle v, q\rangle}{\|v\|\|q\|} = 1$. Recall the Cauchy–Schwarz inequality:

$$|\langle v, q\rangle| \leq \|v\| \cdot \|q\|$$

with equality *if and only if* v and q are linearly dependent, i.e., $v = \alpha q$ for some scalar α. Since $\frac{\langle v, q\rangle}{\|v\|\|q\|} = 1$ (not -1), the scalar must be positive ($\alpha > 0$), so v and q point in the same direction. Intuitively, the only way to achieve a cosine similarity of exactly 1 is for the two vectors to be perfectly aligned — any deviation would reduce the inner product below $\|v\|\|q\|$.

Back to Contents

Question 10.

(a) Expanding:

$$\begin{aligned}
\|\hat{v} - \hat{q}\|^2 &= \langle \hat{v} - \hat{q},\ \hat{v} - \hat{q} \rangle \\
&= \langle \hat{v}, \hat{v} \rangle + \langle \hat{q}, \hat{q} \rangle - 2\langle \hat{v}, \hat{q} \rangle \\
&= \underbrace{\|\hat{v}\|^2}_{=1} + \underbrace{\|\hat{q}\|^2}_{=1} - 2\langle \hat{v}, \hat{q} \rangle \\
&= 2 - 2\langle \hat{v}, \hat{q} \rangle \\
&= 2 - 2\,\frac{\langle v, q \rangle}{\|v\|\|q\|} \\
&= 2 - 2\cos(v, q).
\end{aligned}$$

(b) Since $\|\hat{v} - \hat{q}\|^2 = 2 - 2\cos(v, q)$, minimizing squared Euclidean distance between normalized vectors is equivalent to maximizing cosine similarity (since the 2 is a constant and the -2 flips the direction). A *smaller* distance corresponds to a *larger* cosine similarity. So the two rankings are identical: the post with the highest cosine similarity is also the post with the smallest normalized Euclidean distance.

(c) During training, model parameters (and therefore embedding vectors) are updated via SGD. Each update nudges the embedding vectors of the data instances in the current minibatch. Posts that appear *frequently* in the training set receive many more gradient updates, which tend to push their embeddings farther from the origin (where they were initialized). Posts that appear *rarely* receive few updates, so their embeddings stay close to the initialization (near the origin) and have small norms.

As a result, the magnitude of an embedding vector correlates with how often the corresponding post appeared during training, not with its semantic content. Two posts with identical aesthetics could have very different norms simply because one went viral. This is why normalizing embeddings (or using cosine similarity) is important for semantic retrieval: it strips away the frequency signal and focuses on direction, which better captures meaning.

Question 11.

(a) Euclidean distance is sensitive to vector magnitude. Viral posts have embeddings with $\|v\| \approx 15$, placing them far from the origin in the embedding space. Niche posts cluster near the origin with $\|v\| \approx 2$. When computing $\|v - q\|$, the distance between any two nearby niche posts (including the query) is dominated by the distance *scale* of the niche cluster (≈ 2), while viral posts live in a region with distances on the order of ≈ 15.

However, the problem is that the niche posts — despite being close to each other — may also be close in Euclidean distance to viral posts that happen to be in a similar direction. More critically, the *large norm* of viral posts does

not reflect semantic similarity; it reflects training frequency. The system retrieves viral posts because their large norms skew the distance computation, not because they are semantically close to the query.

(b) Switch to **cosine similarity** (or equivalently, normalize all embeddings to unit norm and then use Euclidean distance). Cosine similarity compares only the *direction* of vectors, ignoring magnitude. This removes the bias toward viral posts whose large norms are an artifact of training frequency, and ensures that retrieval is based on semantic content similarity.

(c) The full retrieval pipeline:

1. **Embed:** Given a raw post x, compute its embedding $h = f(x) \in \mathbb{R}^d$ using the trained embedding model.
2. **Normalize:** Compute the unit-norm embedding $\hat{h} = h/\|h\|$.
3. **Compute similarity:** For each candidate post v_i in the retrieval set (also pre-normalized to $\hat{v}_i$), compute cosine similarity $\cos(\hat{h}, \hat{v}_i) = \langle \hat{h}, \hat{v}_i \rangle$.
4. **Retrieve top-K:** Rank candidates by similarity in descending order. Return the K candidates with the highest cosine similarity:

$$r_k = \arg\max_{v \in D_{k-1}} \langle \hat{v}, \hat{h} \rangle, \qquad D_k = D_{k-1} \setminus \{r_k\}.$$

Question 12.

(a) The NotebookLM pipeline maps directly to this chapter:

- **Embedding function f:** A pre-trained language model (e.g., a sentence transformer) serves as $f : \mathcal{X} \to \mathbb{R}^d$. It converts both document chunks and the user's question into vectors in the same space: $h_i = f(d_i)$ for each chunk, and $h_q = f(q)$ for the question.
- **Distance/similarity measure:** Cosine similarity or inner product after normalization, as in Q8–Q10.
- **Retrieval procedure:** This is exactly the top-K nearest-neighbor retrieval from Q6. The system iteratively finds the most similar chunk, removes it from the candidate set, and repeats:

$$r_k = \arg\max_{v \in D_{k-1}} \mathrm{sim}(v, h_q), \qquad D_k = D_{k-1} \setminus \{r_k\}.$$

The retrieved chunks are then fed to the language model alongside q — this final generation step goes beyond Chapter 4, but the retrieval itself is the pipeline from Q11(c): embed, normalize, compute similarity, retrieve top-K.

(b) Cosine similarity (or equivalently, normalize all embeddings to unit norm and use Euclidean distance). The reasoning from Q8–Q10 applies directly: the

embedding model was trained on a large corpus where common topics (like linear regression) appeared far more frequently than niche ones (like a specialized theorem). As discussed in Q10(c), chunks about common topics receive more gradient updates during training, pushing their embeddings farther from the origin and giving them larger norms. Niche topics, seen rarely, end up with smaller norms. This norm difference reflects training frequency, not relevance to the user's question. Euclidean distance would be biased toward retrieving chunks about common topics regardless of whether they answer the question. Cosine similarity compares only the *direction* of vectors, focusing on semantic content and ignoring the norm artifact.

(c) No, increasing K does not always improve answer quality. There is a trade-off:

- **Too small K:** The system may miss relevant chunks, causing the model to give an incomplete or incorrect answer (e.g., missing a key definition from your lecture notes).
- **Too large K:** The retrieved set will include increasingly irrelevant chunks (those farther from h_q). These add noise that can confuse the language model — for example, retrieving a chunk about optimization when the question is about SVD. Language models also have a finite context window, so injecting too many chunks may exceed the input limit or dilute the signal from the truly relevant ones.

The optimal K balances recall (retrieving all relevant chunks) against precision (avoiding irrelevant ones). In practice, K is treated as a hyperparameter tuned on a validation set.

Back to Contents

5 Chapter 5: Model Ensembling and Generalization — Solutions

Question 1.

Answer: A.

A. *True.* The model overfits: it learns patterns specific to the training data (e.g., particular patient demographics, seasonal illness patterns, or hospital-specific coding practices) that do not generalize to new patients. The gap between training and real-world accuracy is the hallmark of overfitting.

B. *False.* More training would likely *widen* the gap, not close it. The model would memorize training data even more, further hurting generalization.

C. *False.* The training accuracy is indeed a Monte Carlo estimate, but that makes it an *approximation*, not exact. More importantly, the gap is due to overfitting, not estimation error.

D. *False.* We assume past and future data come from the same distribution p_X, but any finite training set is only a sample from p_X. The model may exploit quirks of this sample that do not hold in general.

Question 2.

Answer: B.

A. *False.* Even with the same data and optimizer, different random seeds lead to different random initializations, different minibatch orderings, and different stochastic gradient noise. These collectively produce different final parameters.

B. *True.* The final parameters $\hat{\theta}$ depend on two sources of randomness: (1) the training set D sampled from p_X, and (2) the stochastic optimization noise ϵ (random initialization, minibatch sampling, dropout, etc.). Here D is fixed, so the difference comes entirely from ϵ.

C. *False.* Both colleagues use the same D. The randomness from D would matter if they used different training sets.

D. *False.* Different seeds produce different but not necessarily worse models. In fact, this variation is exactly what enables ensemble methods.

Question 3.

(a)

$$\hat{y} = \frac{1}{3}(0.72 + 0.85 + 0.68) = \frac{2.25}{3} = 0.75.$$

Back to Contents

The ensemble predicts a 75% probability that the disease is present.

(b) The expectation $\mathbb{E}_{D,\epsilon}[f(x, \hat{\theta}(D, \epsilon))]$ requires integrating over all possible training sets D (drawn from the unknown p_X) and all possible optimization noise ϵ. We do not know p_X analytically, and the relationship between (D, ϵ) and $\hat{\theta}$ is defined by the entire training procedure (a complex, nonlinear, iterative process). There is no closed-form expression for this expectation, so we must approximate it with a finite number of samples (Monte Carlo).

(c) Yes, increasing M generally improves the ensemble prediction. By the law of large numbers, the Monte Carlo average $\frac{1}{M}\sum_m f(x, \hat{\theta}^m)$ converges to the true expectation $\mathbb{E}_\epsilon[f(x, \hat{\theta})]$ as $M \to \infty$ (holding D fixed). The practical constraint is computational cost: each model requires a full training run, which can take hours or days. Training 100 models means 100× the compute budget.

Question 4.

(a)

$$c(D; \hat{\theta}) = \frac{1}{10}(0.3+0.1+0.8+0.2+0.5+0.4+0.9+0.1+0.6+0.3) = \frac{4.2}{10} = 0.42.$$

(b) The bootstrap sample $D^1 = \{x_3, x_1, x_5, x_5, x_2, x_8, x_1, x_4, x_3, x_{10}\}$ has costs:

$$\{0.8, 0.3, 0.5, 0.5, 0.1, 0.1, 0.3, 0.2, 0.8, 0.3\}.$$

$$c(D^1; \hat{\theta}) = \frac{1}{10}(0.8+0.3+0.5+0.5+0.1+0.1+0.3+0.2+0.8+0.3) = \frac{3.9}{10} = 0.39.$$

(c) With the set $C = \{c(D^1; \hat{\theta}), \ldots, c(D^{1000}; \hat{\theta})\}$, you can:

1. **Draw a histogram** of the bootstrap costs to visualize the distribution of expected performance.
2. **Compute empirical quantiles** — e.g., the 2.5th and 97.5th percentiles give a 95% confidence interval for the average cost, telling you the range within which the model's performance is likely to fall on future data.
3. **Compute the mean and standard deviation** of the bootstrap costs to summarize central tendency and spread.

Question 5.

(a) The formula measures the overlap between the set of optimal parameters for D and the set of optimal parameters for D'. The numerator is the intersection (parameters that are optimal for *both*), and the denominator is the union (parameters optimal for *either*). The ratio being less than 1 means the two cost functions have different minima — what is optimal for the training data is not optimal for unseen data.

Back to Contents

This implies we should not push training loss to its absolute minimum, because doing so finds a θ that is specifically tuned to D's quirks rather than the underlying pattern. The further we minimize $c(D;\theta)$ beyond the shared optimum, the more we specialize to D and the worse we perform on D'.

(b) You should stop training around **epoch 35–40** (approximately where the validation cost reaches its minimum). This technique is called **early stopping**. You use the **validation set** to decide when to stop — not the training set (which always decreases) and not the test set (which must remain untouched until final evaluation).

(c) Combining training and validation sets *might work* in the **asymptotic regime** — when the dataset is so large that a finite sample is essentially representative of the full distribution p_X. In that case, the minima of cost functions defined on different subsets largely overlap, and more data simply helps.

It would *fail* when the dataset is small or moderate-sized. In this case, the minima on different subsets diverge, and without a held-out validation set, you have no way to detect overfitting or decide when to stop training. You would blindly minimize the training cost, likely overfitting.

Question 6.

(a)

- **Training set:** Used to find a good parameter configuration $\hat{\theta}$ by minimizing the empirical cost via SGD.
- **Validation set:** Used to select the best model checkpoint (early stopping) and tune hyperparameters (learning rate, architecture choices, etc.).
- **Test set:** Used *once* at the very end to get an honest, unbiased estimate of how the model will perform on future, unseen data. Never used for any decision-making during training or model selection.

(b) The validation set was used to *select* the model (e.g., choosing the best epoch, tuning the learning rate). This means the model was implicitly optimized to perform well on the validation set. Reporting validation accuracy as "expected real-world performance" is optimistic — it overestimates how well the model will do on truly unseen data. The validation set has effectively become a second training set for hyperparameter selection. Only the test set, which was never used for any decision, provides an unbiased performance estimate.

(c) The three sets must be **mutually disjoint**: $D_{\text{train}} \cap D_{\text{val}} = \emptyset$, $D_{\text{train}} \cap D_{\text{test}} = \emptyset$, and $D_{\text{val}} \cap D_{\text{test}} = \emptyset$. If any overlap exists, the performance estimates become biased. For example, if training and test sets share data points, the "test" accuracy reflects how well the model memorized those shared points, not how well it generalizes.

Question 7.

Back to Contents

(a) With $|D_0| = 6$, LOOCV trains the model **6 times**. In each round n, the training set is $D_0 \setminus \{x_n\}$ (5 cases) and the test set is $\{x_n\}$ (1 case).

(b) The LOOCV estimate is the average of the per-instance costs:

$$\hat{c}_{\text{LOOCV}} = \frac{1}{6}(0.3 + 0.7 + 0.2 + 0.5 + 0.4 + 0.6) = \frac{2.7}{6} = 0.45.$$

(c) One possible partition for 3-fold CV:

- **Fold 1:** Test on $\{x_1, x_2\}$, train on $\{x_3, x_4, x_5, x_6\}$.
- **Fold 2:** Test on $\{x_3, x_4\}$, train on $\{x_1, x_2, x_5, x_6\}$.
- **Fold 3:** Test on $\{x_5, x_6\}$, train on $\{x_1, x_2, x_3, x_4\}$.

You train the model **3 times** (once per fold). The final performance estimate averages the costs across all three folds.

(d) No. With millions of records across hospital networks, LOOCV would require training the model *millions* of times — one full training run per data point. This is computationally infeasible. With such a large dataset, a single train/validation/test split provides a sufficiently reliable performance estimate, since each partition is large enough to be representative of p_X.

Question 8.

(a) With 95% confidence, the model's average cost on future data (drawn from the same distribution) will fall between 0.28 and 0.52. This means: if we deployed the model to many independent batches of new patients, about 95% of the time the average cost over each batch would be in this range. This gives the Chief of Medicine a concrete, quantified statement about expected performance and its uncertainty.

(b) Computing the true distribution of the average cost would require knowing the full data distribution p_X — the probability of every possible patient presentation. This is unknown and extremely complex (it depends on demographics, genetics, environmental factors, etc.). Without p_X, we cannot compute $\mathbb{E}_{D'}[c(D'; \theta)]$ or its distribution analytically. Bootstrapping sidesteps this by using the empirical distribution (the test set) as a proxy for p_X and resampling from it.

Question 9.

(a) The left type is called **epistemic** (or **reducible**) noise. The right type is called **aleatoric** (or **irreducible**) noise.

(b) Epistemic noise can be reduced by ensembling. Each model in the ensemble produces a slightly different prediction due to different optimization randomness ϵ (different random seeds, minibatch orderings, etc.). These differences are essentially "noise" from the learning process. By averaging M predictions,

this noise is reduced by a factor of roughly $\frac{1}{\sqrt{M}}$ (by the central limit theorem). The ensemble's prediction converges to the expected prediction, smoothing out the randomness from any single training run.

(c) This is **aleatoric (irreducible) noise**. The overlapping symptoms are inherent in the biology of these diseases and have nothing to do with the model or training process. No amount of training data, better architectures, or ensembling can distinguish bacterial from viral pneumonia when the symptom profiles are identical. Even a perfect model with infinite data would still get this wrong, because the outcome is genuinely ambiguous from the symptom profile alone — the same input maps to different true labels.

Question 10.

A complete evaluation pipeline:

1. **Data partitioning:** Split D_0 into three disjoint sets — training (e.g., 80%), validation (10%), and test (10%). The training set is used to learn parameters via SGD. The validation set is used for early stopping and hyperparameter tuning. The test set is held out until the very end.

2. **Model selection:** Train the model on the training set while monitoring the cost on the validation set at each epoch. Stop training when the validation cost starts increasing (early stopping). Save the checkpoint with the lowest validation cost. Optionally, tune hyperparameters (learning rate, architecture) by comparing validation performance across configurations.

3. **Performance estimation with uncertainty:** Evaluate the best model on the test set to get a point estimate of future performance. To quantify uncertainty, use **bootstrapping**: resample the test set with replacement M times, compute the average cost for each bootstrap sample, and report the empirical 95% confidence interval (2.5th and 97.5th percentiles). This tells the Chief of Medicine: "With 95% confidence, the model's cost on future patients will be between l and u."

4. **Ensembling:** Train M models with different random seeds (same data, different ϵ). Average their predictions to form an ensemble predictor. This reduces **epistemic noise** — the variability due to stochastic optimization — and typically improves accuracy beyond any single model.

5. **What we can vs. cannot reduce:** Ensembling and more data can reduce **epistemic (reducible) noise** from the learning process. However, **aleatoric (irreducible) noise** — randomness inherent in patient biology (misreported symptoms, atypical presentations, comorbidities) — cannot be eliminated by any model improvement. The confidence interval from bootstrapping reflects both types of noise and gives an honest bound on achievable performance.

Back to Contents

Question 11.

(a) Data contamination violates the requirement that the three sets must be **mutually disjoint**: $D_{\text{train}} \cap D_{\text{test}} = \emptyset$. When benchmark questions leak into the training data, $D_{\text{train}} \cap D_{\text{test}} \neq \emptyset$. The "test" accuracy no longer measures generalization to unseen data — it partially measures how well the model memorized training examples that happen to also be in the benchmark.

(b) The 92% score is analogous to a *training* accuracy, not a test accuracy: the model has seen those exact questions during training and may have memorized them. Just like in Q1, where training accuracy (95%) was much higher than real-world accuracy (78%), the contaminated model's high benchmark score reflects overfitting to those specific questions, not genuine understanding. The uncontaminated model's 81% is a more honest estimate of real-world performance. You cannot compare the two scores on equal footing because they measure different things: one measures memorization, the other measures generalization.

(c) This is an open-ended question — many valid strategies exist. Any answer that identifies a concrete mitigation and connects it to a concept from this chapter receives full credit. Some example strategies:

1. **Create new, private benchmarks regularly.** If the test set is kept secret and refreshed periodically, it cannot leak into training data. This enforces the disjointness requirement ($D_{\text{train}} \cap D_{\text{test}} = \emptyset$) by construction.

2. **Use contamination detection and reporting.** Before evaluating, check whether benchmark questions appear in the training data (e.g., via n-gram overlap). Report contamination rates alongside scores so reviewers can adjust their interpretation. This is analogous to the principle from Q6 that the test set must never influence any decision-making — if contamination is detected, those scores should be discarded.

3. **Encrypt or perturb benchmark data before public release** so that exact memorization is impossible, while the evaluation task remains equivalent.

4. **Use held-out human evaluation** on novel, dynamically generated prompts, bypassing fixed benchmarks entirely.

Other reasonable strategies are acceptable as long as they (i) clearly describe the mitigation and (ii) tie it back to a concept from this chapter (e.g., disjointness, overfitting, generalization gap).

Question 12.

(a) Similarities: Both MoE and the ensemble from Q3 combine the outputs of multiple models into a single prediction via a weighted average. Both aim to improve predictions beyond what any single model can achieve.

Differences:

- In Q3's ensemble, *all* M models see every input and contribute equally (weights $= \frac{1}{M}$). In MoE, only the top-K experts are activated per input — the rest are skipped entirely.
- In Q3's ensemble, the weights are uniform ($\frac{1}{M}$) and fixed. In MoE, the weights $g_m(x)$ are *input-dependent* — the router learns to assign different experts to different types of inputs. A vocabulary question about food might activate different experts than one about grammar.
- In Q3's ensemble, each model is trained independently with different random seeds. In MoE, all experts are trained jointly as part of one model, with the router learned end-to-end.

(b) No, MoE does not achieve the same $\frac{1}{\sqrt{M}}$ noise reduction. That result relies on averaging M *independent* predictions — the law of large numbers requires independence (or at least low correlation) between the models. In MoE, only $K \ll M$ experts contribute to each prediction, so you are averaging over K models, not M. Furthermore, the experts are trained jointly (not independently), so their errors may be correlated. The noise reduction from MoE comes from *specialization* (different experts handle different input types well) rather than from the statistical averaging effect of ensembling.

(c) In a standard ensemble from Q3, every model must process every input: computing $\hat{y} = \frac{1}{M}\sum_{m=1}^{M} f(x;\hat{\theta}^m)$ requires M full forward passes. If each model takes time T, the ensemble takes MT. In MoE, the router selects only K experts, so only K forward passes are needed (plus the lightweight router computation). Since $K \ll M$ (e.g., $K = 2$ out of $M = 8$), MoE achieves much of the benefit of having M specialized models while paying only K/M of the computational cost. This is why MoE models can have very large total parameter counts while remaining efficient at inference time.

Back to Contents

6 Chapter 6: Dimensionality Reduction and Regularization — Solutions

Question 1.

Answers: A, B, and C.

A. *True.* SVD projects data onto a linear subspace (a hyperplane through the origin). The projection is a matrix multiplication $W^\star x$, which is a linear operation.

B. *True.* SVD requires $\langle w_i, w_j \rangle = 0$ for $i \neq j$. The singular vectors must be mutually orthogonal.

C. *True.* SVD requires $\|w_i\| = 1$ for all i. Each singular vector must have unit norm.

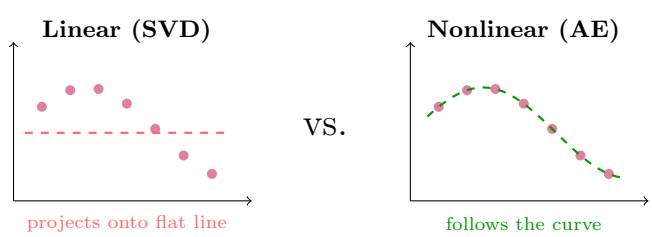
D. *False.* SVD has a closed-form solution via eigendecomposition of $A^\top A$. It does not require gradient descent.

E. *False.* SVD minimizes reconstruction error subject to the constraints, but the error is not zero unless $K = d$ (i.e., no dimensionality reduction).

Question 2.

(a) If both F and G are linear (i.e., $F(x) = Ux$ and $G(z) = Vz$ for matrices $U \in \mathbb{R}^{K \times d}$ and $V \in \mathbb{R}^{d \times K}$), then minimizing $\|x - VUx\|^2$ recovers the same subspace as SVD (though without the orthogonality and unit-norm constraints on U and V individually). The optimal linear autoencoder spans the same K-dimensional subspace as the top-K singular vectors.

With nonlinear activations (e.g., ReLU, tanh), the autoencoder can learn curved, nonlinear mappings. This allows it to capture how scent molecules interact nonlinearly — for instance, vanillin and musk may combine to produce a "cozy" note that is not a linear combination of their individual contributions. SVD, being linear, would miss such interactions entirely.

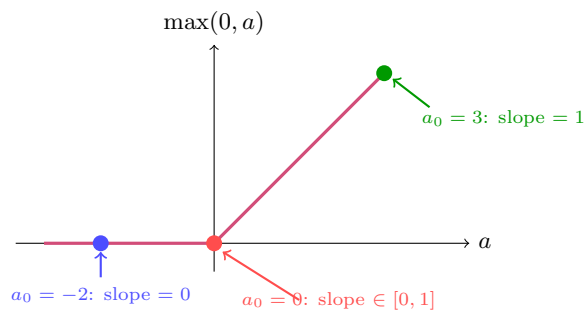


(b) It is called an "autoencoder" because it *encodes* and *decodes* the *same* instance. The scent profile x is encoded to a compact latent vector $z = F(x)$, then decoded back to a reconstructed profile $\hat{x} = G(z)$. The "auto" prefix

indicates that the target output is the input itself — the network learns to reconstruct its own input through a bottleneck.

Question 3.

(a) The sub-derivative of $f(a) = \max(0, a)$:



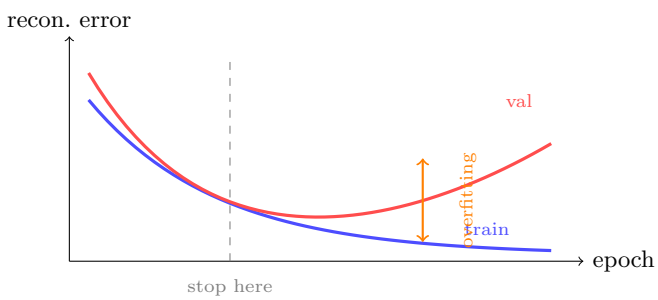
- At $a_0 = -2$: $f(a) = 0$ in a neighborhood of $a_0 = -2$ (constant function). Both left and right limits of the difference quotient are 0. Sub-derivative: $[0, 0] = \{0\}$.
- At $a_0 = 3$: $f(a) = a$ in a neighborhood of $a_0 = 3$ (identity function). Both limits are 1. Sub-derivative: $[1, 1] = \{1\}$.
- At $a_0 = 0$: The left limit uses the constant branch ($f(a) = 0$ for $a < 0$), giving $l = 0$. The right limit uses the linear branch ($f(a) = a$ for $a > 0$), giving $r = 1$. Sub-derivative: $[0, 1]$. Any value in this interval is a valid sub-gradient.

(b) In practice, the probability that any pre-activation value is *exactly* zero (to machine precision) is essentially zero, since the inputs and weights are continuous-valued. The non-differentiable point is a set of measure zero and is almost never encountered during training. PyTorch uses 0 as the derivative at $a_0 = 0$ (i.e., it treats ReLU as $f'(0) = 0$), though any value in $[0, 1]$ would work since this case almost never arises.

Question 4.

(a) Without regularization, a high-capacity autoencoder can learn the *identity function*: it memorizes each training perfume's profile exactly (z simply encodes a lookup index) without learning any meaningful compressed representation. The reconstruction error on training data would be near zero, but the latent vector z would carry no useful structure — it would not generalize to new perfumes, and the 2D scent map would be meaningless. The autoencoder overfits to the training set rather than discovering the underlying scent manifold.

Back to Contents



(b) Two methods of regularization from the chapter:

1. **Early stopping:** Monitor the reconstruction error on a held-out validation set during training. Stop when the validation error starts increasing, even if the training error continues to decrease. This prevents the autoencoder from memorizing the training data and ensures the learned scent structure generalizes.

2. **Input corruption (denoising):** Corrupt each scent profile with noise (Gaussian or masking) before encoding, but train the decoder to reconstruct the *clean* profile. This forces the encoder to learn robust features that capture the true scent structure rather than memorizing exact molecule concentrations. The model cannot simply memorize inputs because it never sees the clean version as input during training.

Question 5.

(a) The two conditions:

1. **Semantic equivalence:** The corrupted input $s(x, \epsilon)$ should be "close enough" to x that they represent the same scent. Adding a small amount of Gaussian noise to molecule concentrations does not change what the perfume smells like — a perfume with 3.02 units of linalool instead of 3.00 is essentially the same fragrance.

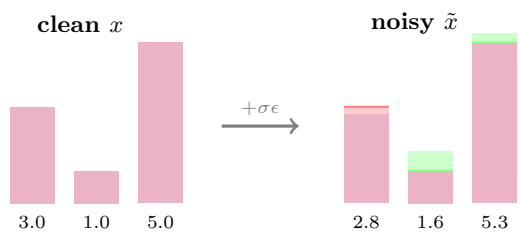
2. **Full support:** For any point $x' \in \mathbb{R}^d$, the probability $p(s(x, \epsilon) = x')$ must be non-zero. The standard Normal distribution has full support (it assigns non-zero probability density to every real number), so $x + \sigma\epsilon$ can in principle produce any point in $\mathbb{R}^d$, satisfying this condition.

Gaussian noise satisfies semantic equivalence *under the assumption that the data is invariant to small local perturbations* — i.e., small changes in molecule concentrations do not change the perceived scent. This is natural for chemical profiles but may not hold for all data types.

(b)

$$\tilde{x} = x + \sigma\epsilon = \begin{bmatrix} 3.0 \\ 1.0 \\ 5.0 \end{bmatrix} + 0.5 \begin{bmatrix} -0.4 \\ 1.2 \\ 0.6 \end{bmatrix} = \begin{bmatrix} 3.0 - 0.2 \\ 1.0 + 0.6 \\ 5.0 + 0.3 \end{bmatrix} = \begin{bmatrix} 2.8 \\ 1.6 \\ 5.3 \end{bmatrix}.$$

Back to Contents



(c) The model is trained to reconstruct the *clean* x, not the noisy $\tilde{x}$, because the goal is to learn the underlying scent structure, not the measurement noise. If the target were the noisy input, the model would learn to reproduce noise — it would be rewarded for outputting noisy reconstructions, which defeats the purpose of denoising. By requiring reconstruction of the clean profile, the model must learn to *filter out* noise and extract the true scent signal, which forces it to capture robust, generalizable features.

Question 6.

(a) No, masking alone does *not* satisfy full support. The corrupted input $\tilde{x} = m \odot x$ can only produce vectors where each dimension is either x_i (if $m_i = 1$) or 0 (if $m_i = 0$). There are only 2^d possible outputs, all of which are constrained to have entries that are either 0 or the original value. This is a finite, discrete set, not all of $\mathbb{R}^d$.

This is typically fixed by filling the zeroed-out dimensions with random noise (e.g., drawn from a standard Normal distribution) instead of leaving them at zero. This restores full support, since the random fill-in values can take any real value.

(b) Masking assumes that the data is invariant to dropping individual dimensions (semantic equivalence). For chemical profiles, dropping a few molecule readings does not change the overall scent category — a perfume is still "woody" even if you lose the citral measurement. For text, however, dropping a single word can completely change the meaning. For example:

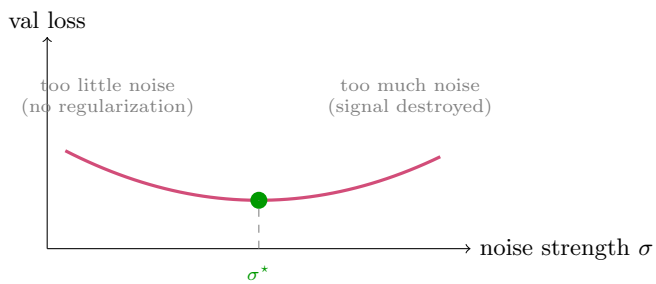
- "This perfume does *not* contain allergens" → "This perfume does contain allergens" (dropping "not" reverses the safety claim).

The semantic equivalence condition is violated: the corrupted sentence has the opposite meaning of the original.

Discussion note: If masking violates semantic equivalence (and full support), why does BERT work so well? Three reasons: (1) these conditions are *sufficient*, not necessary — violating them removes the theoretical guarantee but does not mean the model fails; (2) BERT's objective is narrower than a denoising autoencoder: it only predicts the masked tokens from context, rather than reconstructing the entire input; (3) BERT hedges by only masking 15% of tokens, and of those, 80% become `[MASK]`, 10% become a random token (partially restoring full support in the discrete vocabulary), and 10% stay unchanged.

Back to Contents

(c) Both σ and μ are hyperparameters that control the strength of corruption. You choose them by trying several values and selecting the one that gives the lowest **validation loss** (reconstruction error on a held-out validation set). Too little noise ($\sigma \approx 0$ or $\mu \approx 1$) provides no regularization benefit. Too much noise (σ very large or $\mu \approx 0$) destroys so much information that the model cannot reconstruct the clean profile.



Question 7.

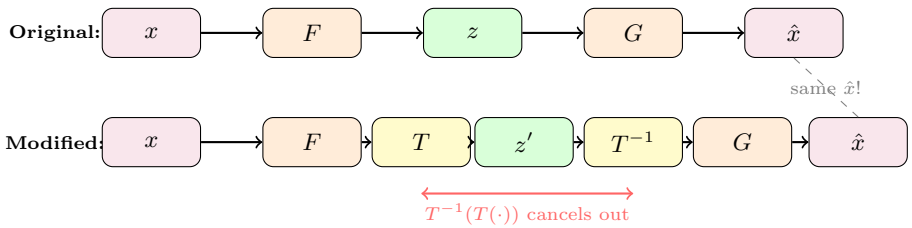
(a) Answer: B. SVD can only project onto linear (flat) subspaces. When perfumes lie along a curved scent spectrum (e.g., fresh → floral → woody is not a straight line in molecule space), a linear projection collapses points that are far apart on the manifold onto nearby points in the 2D space, destroying the ordering. SVD cannot "unfold" the curve.

(b) The deterministic autoencoder minimizes reconstruction error without regularization. While it can learn nonlinear mappings (unlike SVD), it may overfit to the training data: the encoder might memorize specific training perfumes rather than learning the smooth underlying scent manifold. The denoising autoencoder, by contrast, is trained to reconstruct clean profiles from noisy ones. This forces the encoder to learn *robust* features that capture the true scent structure and are invariant to small perturbations. The result is a smoother, more meaningful latent space — nearby perfumes on the scent manifold are consistently mapped to nearby points in the 2D map, rather than in an arbitrary arrangement that happens to minimize training reconstruction error.

Question 8.

(a) Identifiability means that the model parameters can be uniquely determined from the data. If the model is identifiable, then two different parameter settings θ and θ' that produce the same input-output behavior must in fact be the same: $\theta = \theta'$.

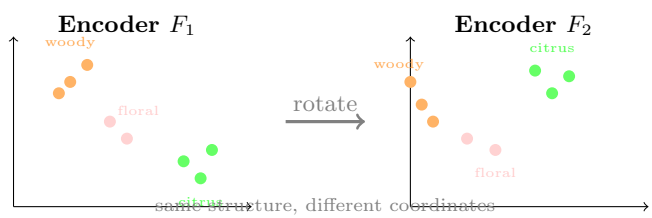
Nonlinear autoencoders are typically *not* identifiable because there are infinitely many ways to compose nonlinear functions that produce the same overall mapping. For instance, if F and G achieve a certain reconstruction error, we can insert an arbitrary invertible transformation T between them:



$F' = T \circ F$ and $G' = G \circ T^{-1}$ gives $G'(F'(x)) = G(T^{-1}(T(F(x)))) = G(F(x))$ — the same reconstruction, but a completely different latent vector $z' = T(z)$. The flexibility of nonlinear computation means infinitely many (θ_F, θ_G) pairs achieve the same error.

(b) You should be cautious about drawing strong conclusions from the scent map. The specific arrangement of perfumes in 2D (which clusters appear, where they are located, how they are shaped) depends on the particular F that was found, which is one of many equally valid solutions. A different random seed could produce a completely different-looking map with the same reconstruction quality. Patterns you observe (e.g., "woody orientals form a tight cluster") might be real, or they might be artifacts of the particular solution found. You should scrutinize patterns before concluding they reflect true structure in the fragrance data.

(c) If $z_2 = Rz_1$ for a rotation matrix R, the 2D scent map is simply rotated — the relative distances and cluster structure are fully preserved. No information is lost or gained by the rotation.



This means even though $z_1 \neq z_2$ (the model is not identifiable in the strict sense), the two representations are equivalent for the purpose of visualization and analysis.

This relaxed notion of identifiability is more practical because in most applications, we care about the *structure* of the latent space (which perfumes are close together, which scent families cluster) rather than the specific coordinate values. If we can guarantee identifiability up to rotations (or other structure-preserving transformations), that is sufficient for meaningful analysis.

Question 9.

Answer: C.

A. *Not best.* SVD is fast but can only capture linear structure. Scent molecules interact nonlinearly, so SVD would miss important patterns (e.g., how vanillin and musk together create a "cozy" note that neither has alone).

B. *Not best.* Without regularization, the autoencoder can overfit and learn a meaningless representation (e.g., memorizing training perfumes rather than learning scent structure).

C. *Best.* A denoising autoencoder with early stopping combines two forms of regularization: input corruption forces robust feature learning, and early stopping prevents overfitting. Choosing σ via validation loss ensures the noise level is appropriate for the data.

D. *Not best.* While not unreasonable, applying SVD first discards nonlinear structure that the autoencoder could have captured. It is better to let the autoencoder work directly on the high-dimensional scent profiles so it can discover nonlinear molecule interactions.

Question 10.

Three (or more) reasons the intern's conclusion is too strong:

1. **Non-identifiability:** Nonlinear autoencoders are not identifiable. A different random seed, initialization, or training run could produce a completely different 2D scent map with the same reconstruction quality. The specific clusters the intern sees may be one of many equally valid arrangements — they are not the unique "true" structure of the fragrance space. The clusters might rearrange, merge, or split under a different training run.

2. **Information loss from $K = 2 \ll K'$:** The true dimensionality of interesting scent variations is almost certainly $K' \gg 2$. Fragrance families are distinguished by many independent axes: freshness, sweetness, smokiness, floral character, sillage, etc. Projecting to $K = 2$ inevitably loses information. The autoencoder must implicitly choose which 2 out of K' dimensions of variation to preserve, and this choice is determined by the architecture, regularization, and training randomness. The apparent "woody oriental vs. fresh citrus" separation may be real, but other important distinctions (e.g., gourmand vs. aquatic within the fresh category) are necessarily lost and invisible in the 2D map.

3. **Implicit choices from architecture, regularization, and training:** The scent map is shaped by many implicit decisions: network depth and width, the type and strength of corruption (σ or μ), early stopping epoch, optimizer hyperparameters, and random seed. Different choices could highlight different aspects of the fragrance data. The intern's visualization reflects one particular combination of these choices, not an objective

ground truth. The claim that the autoencoder "perfectly separates fragrance families" is especially suspect since the separation could be an artifact of these choices rather than a fundamental property of the data.

The correct approach is to treat the visualization as *suggestive* and corroborate any observed patterns with additional analysis (e.g., checking whether the clusters hold across multiple training runs, validating with expert fragrance classifications, or testing whether similar scent profiles yield similar perceived notes).

Question 11.

(a) BERT uses the same *corrupt-then-reconstruct* principle as a denoising autoencoder, with masking as the corruption:

- $x =$ the original sequence of tokens (e.g., "The perfume smells like vanilla").
- $\tilde{x} = m \odot x =$ the masked sequence (e.g., "The perfume [MASK] like vanilla"), where $\sim$15% of tokens are replaced with a special [MASK] token.
- F (encoder) = the Transformer encoder, which reads the corrupted sequence and produces a contextual representation z for each token position.
- G (decoder) = a prediction head on top of the encoder that maps each masked position's representation back to a probability distribution over the vocabulary, aiming to reconstruct the original token.

An important distinction: BERT is not strictly a denoising autoencoder because it lacks a *dimensional bottleneck* — the hidden representation has the same (or larger) dimension as the input embedding, so there is no compression through a narrow layer. Instead, the information bottleneck comes from the masking itself: the model must reconstruct missing tokens using only the surrounding context. The architectural parallel is the corruption strategy, not the bottleneck.

(b) A low masking rate (15%) preserves semantic equivalence because most of the sentence remains intact — the surrounding context still conveys the same meaning, and a human (or model) can infer the missing words from context. This satisfies the chapter's first condition: $\tilde{x}$ and x are semantically equivalent.

If BERT masked 80% of tokens, most of the sentence would be destroyed. The corrupted input "[MASK] [MASK] [MASK] like [MASK]" is not semantically equivalent to the original — there is not enough context to determine the meaning, let alone reconstruct the missing words. The model would be forced to "guess" rather than reason from context, and the learned representations would be poor. This is analogous to the chapter's discussion of choosing μ via validation loss: too much corruption destroys the signal.

(c) Discrete masking replaces tokens with a `[MASK]` symbol — the corrupted input lives in a finite set of possible masked sequences. Gaussian noise on

continuous embeddings instead perturbs each embedding vector by $\tilde{e} = e + \sigma\epsilon$, $\epsilon \sim \mathcal{N}(0, I)$, producing a corrupted embedding that can be *any* point in $\mathbb{R}^d$.

This automatically satisfies the **full support** condition from Q5: the Gaussian distribution has nonzero density everywhere, so the corrupted embedding can in principle land anywhere in the continuous embedding space. Discrete masking does *not* satisfy full support (as discussed in Q6a) because the corrupted input is restricted to the finite set of masked sequences. Moving to continuous noise removes this limitation while preserving the corrupt-then-reconstruct training principle.

(d) Both models demonstrate that corruption-based training can produce powerful general-purpose representations *without any labels*. The corruption itself acts as the supervisory signal: the model must understand the underlying structure of the data well enough to "fill in" what was corrupted.

This connects directly to the chapter's core message: naively minimizing reconstruction error on clean data (a deterministic autoencoder) can lead to memorization and poor generalization. Adding corruption *forces* the model to learn robust, generalizable features rather than superficial input-output mappings. The progression from discrete masking (BERT) to continuous Gaussian corruption (DiffusionBERT) shows that the denoising principle is not tied to any particular corruption mechanism — it is a general strategy, and the theoretical conditions from this chapter (semantic equivalence, full support) explain why some corruption choices work better than others.

7 Chapter 7: Clustering — Solutions

Question 1.

Answers: A and B.

A. *True.* A hard assignment requires each entry to be exactly 0 or 1 — the student is either in a house or not.

B. *True.* The entries must sum to 1, ensuring the student is assigned to exactly one house. Together with A, this makes z_n a one-hot vector.

C. *Not sufficient.* Real values in $[0, 1]$ summing to 1 describe a *soft* assignment (probability distribution), not a hard one-hot vector. For example, $z = (0.3, 0.3, 0.2, 0.2)$ satisfies C and B but is not a valid hard assignment.

D. *False.* In k-means clustering, each data point belongs to exactly one cluster. Belonging to two houses simultaneously would violate the one-hot constraint.

E. *True but redundant.* Any one-hot vector automatically has $\|z_n\| = \sqrt{0^2 + \cdots + 1^2 + \cdots + 0^2} = 1$, so this is a consequence of A and B, not an independent defining constraint. Moreover, $\|z\| = 1$ alone does not imply one-hot (e.g., $z = (0.5, 0.5, 0.5, 0.5)$ has $\|z\| = 1$ but is not one-hot). The defining constraints are A and B.

Question 2.

(a) For student $x = (6, 5)$:

$$\begin{aligned}
\|x - \mu_G\|^2 &= (6-8)^2 + (5-2)^2 = 4 + 9 = 13 \\
\|x - \mu_S\|^2 &= (6-3)^2 + (5-9)^2 = 9 + 16 = 25 \\
\|x - \mu_R\|^2 &= (6-7)^2 + (5-7)^2 = 1 + 4 = \mathbf{5} \\
\|x - \mu_H\|^2 &= (6-2)^2 + (5-3)^2 = 16 + 4 = 20
\end{aligned}$$

The smallest distance is 5 (Ravenclaw). The student is assigned to **Ravenclaw**: $z = (0, 0, 1, 0)^\top$.

(b) For student $x' = (5, 8)$:

By visual inspection, x' is close to both Slytherin $(3, 9)$ and Ravenclaw $(7, 7)$. Computing:

$$\begin{aligned}
\|x' - \mu_G\|^2 &= (5-8)^2 + (8-2)^2 = 9 + 36 = 45 \\
\|x' - \mu_S\|^2 &= (5-3)^2 + (8-9)^2 = 4 + 1 = \mathbf{5} \\
\|x' - \mu_R\|^2 &= (5-7)^2 + (8-7)^2 = 4 + 1 = \mathbf{5} \\
\|x' - \mu_H\|^2 &= (5-2)^2 + (8-3)^2 = 9 + 25 = 34
\end{aligned}$$

Back to Contents

There is a **tie**: Slytherin and Ravenclaw both have distance 5. K-means does not have a principled way to break ties — the assignment depends on implementation (e.g., choose the first one, choose randomly). This is one motivation for soft assignment in MoG, where the student would receive equal posterior probability for both houses.

Question 3.

(a) The centroid is the average of all members:

$$\mu_G = \frac{1}{3}\big((9,3) + (7,1) + (8,5)\big) = \frac{1}{3}(24,9) = (8,3).$$

$$\mu_S = \frac{1}{2}\big((2,10) + (4,8)\big) = \frac{1}{2}(6,18) = (3,9).$$

Note that μ_S did not change (it was already $(3,9)$), while μ_G shifted from $(8,2)$ to $(8,3)$ — the bravery component stayed the same but cunning increased, reflecting Hermione's higher cunning score pulling the centroid upward.

(b) The centroid μ_k represents the "average personality" of house k — the prototypical trait profile of that house. In Hogwarts terms, it is the house's defining character. As new students are sorted and join a house, the house's identity naturally evolves to reflect its current members. Recomputing the centroid after each round ensures the house profile stays up-to-date. For example, if several brave-but-cunning students join Gryffindor, the house centroid shifts to reflect that blend.

Question 4.

Answer: B.

- **A.** *Wrong order.* You cannot update centroids before assigning students to houses — the centroid depends on knowing which students are in each house.
- **B.** *Correct.* K-means alternates between: (1) assigning each student to the nearest centroid, and (2) updating each centroid as the average of its assigned students. This repeats until the assignments no longer change (convergence).
- **C.** *False.* K-means never removes clusters. The number of clusters K is fixed throughout the algorithm.
- **D.** *False.* Random assignment without centroid updates is not an optimization algorithm and would produce meaningless clusters.

Question 5.

(a) Because z_n is a one-hot vector, exactly one entry z_n^k equals 1 and the rest are 0. The sum $\sum_{k=1}^{K} z_n^k \|x_n - \mu_k\|^2$ therefore reduces to a single term: the squared distance to the assigned house's centroid. The other three terms are multiplied by 0 and vanish.

(b) Both students are assigned to Gryffindor, so $z_1^G = z_2^G = 1$. The update:

$$\begin{aligned}\frac{\eta}{M}\sum_{m=1}^{M} z_m^G(x_m - \mu_G) &= \frac{1.0}{2}\Big[1\cdot\big((9,4)-(8,3)\big) + 1\cdot\big((7,2)-(8,3)\big)\Big] \\ &= \frac{1}{2}\Big[(1,1)+(-1,-1)\Big] \\ &= \frac{1}{2}(0,0) = (0,0).\end{aligned}$$

So $\mu_G \leftarrow (8,3) + (0,0) = (8,3)$. The centroid does not move.

(c) The closed-form centroid is $\frac{1}{2}\big((9,4)+(7,2)\big) = (8,3)$, which is the same as the SGD result. This is not a coincidence: when $\eta = 1$ and the minibatch contains *all* members of the cluster, the SGD update with $\eta = 1$ is equivalent to the closed-form update. The gradient points from the current centroid toward the average of the assigned points, and with step size 1, we jump directly to the average. With $\eta < 1$ or a partial minibatch, the SGD update would only move partway toward the closed-form solution.

Question 6.

(a) A uniform prior $p(z = k) = \frac{1}{4}$ means the Sorting Hat has no inherent preference for any house before seeing a student's traits. Each house is equally likely *a priori*. This reflects the assumption that the four houses should be roughly equal in size.

A non-uniform prior example: $p(z = G) = 0.4$, $p(z = S) = 0.1$, $p(z = R) = 0.3$, $p(z = H) = 0.2$. This might be appropriate if historical data shows that more students tend to be Gryffindors (perhaps bravery is more common than cunning in the general population), or if the school wants to maintain specific house sizes for resource allocation.

(b) When σ^2 is **very small**: The Gaussian distributions around each centroid are tight and well-separated. Students near a centroid are assigned to that house with near-certainty. In Hogwarts terms, each house has a very specific personality type, and students clearly belong to one house — the Sorting Hat is decisive and confident.

When σ^2 is **very large**: The Gaussians spread out and overlap heavily. Many students have similar probability of belonging to multiple houses. The Sorting Hat becomes indecisive: "You could do well in Slytherin *or* Gryffindor…" The clusters lose their distinctiveness, and soft assignment becomes necessary since hard boundaries are not meaningful.

Back to Contents

Question 7.

(a) With $\sigma^2 = 1$, the unnormalized log-posteriors are $-\frac{1}{2}d_k$:

House	d_k	$-\frac{1}{2}d_k$
Gryffindor	13	-6.5
Slytherin	25	-12.5
Ravenclaw	5	-2.5
Hufflepuff	20	-10.0

Ravenclaw has the largest (least negative) log-posterior at -2.5, so it dominates. The gap between Ravenclaw and the next closest (Gryffindor at -6.5) is 4.0 in log-space, which means Ravenclaw's unnormalized posterior is $e^4 \approx 55$ times larger. The distribution is highly concentrated on Ravenclaw — this is essentially a hard assignment.

(b) With $\sigma^2 = 100$, the unnormalized log-posteriors are $-\frac{1}{200}d_k$:

House	d_k	$-\frac{1}{200}d_k$
Gryffindor	13	-0.065
Slytherin	25	-0.125
Ravenclaw	5	-0.025
Hufflepuff	20	-0.100

All four values are very close to 0 (and to each other). The maximum gap is only 0.1 in log-space ($e^{0.1} \approx 1.1$), so the posteriors are nearly uniform: roughly $\frac{1}{4}$ for each house. The Sorting Hat is highly uncertain — it cannot distinguish which house the student belongs to.

(c) In the limit $\sigma^2 \to 0$: The posterior concentrates entirely on the house with the smallest distance. All probability mass goes to $k^\star = \arg\min_k \|x - \mu_k\|^2$, and $p(z = k|x) = 0$ for all $k \neq k^\star$. This is exactly the **k-means hard assignment**.

In the limit $\sigma^2 \to \infty$: All the exponents $-\frac{1}{2\sigma^2}d_k \to 0$, so all terms become $e^0 = 1$, and the posterior becomes the **uniform distribution** $p(z = k|x) = \frac{1}{K}$ for all k. The distances become irrelevant and no sorting occurs.

Question 8.

(a) The sum inside the log is necessary because the house assignment z is a *latent* (unobserved) variable. We do not directly observe which house a student truly belongs to — we only observe their trait vector x_n. Since the true house is unknown, we must consider all possible houses and their contributions to the likelihood of observing x_n. The sum $\sum_k p(x_n|z = k)p(z = k)$ is the *total probability* of observing x_n under the model, obtained by marginalizing out the latent variable z. Assigning the student to one house first and then computing the likelihood would ignore the uncertainty about the true assignment and underestimate the total probability.

(b) The posterior $p(z = k|x_n)$ acts as a *soft assignment weight* — it determines how much influence student x_n has on updating centroid μ_k. If $p(z = k|x_n) \approx 1$

(the student almost certainly belongs to house k), the gradient is strong: the centroid is pulled toward the student. If $p(z = k|x_n) \approx 0$ (the student almost certainly does *not* belong to house k), the gradient is near zero: this student has essentially no influence on μ_k. This is more nuanced than k-means, where each student either fully contributes (weight 1) or does not contribute at all (weight 0). In MoG, a student in the "boundary region" between two houses influences both centroids, proportionally to how much it belongs to each.

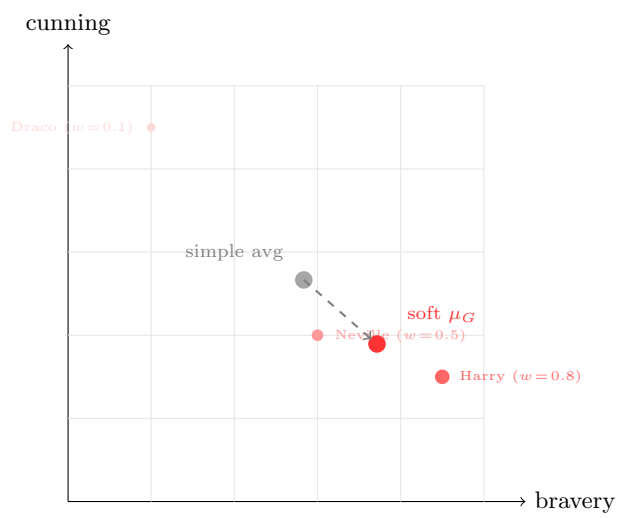
Question 9.

(a) The soft (weighted) centroid:

$$\mu_G = \frac{0.8 \cdot (9,3) + 0.5 \cdot (6,4) + 0.1 \cdot (2,9)}{0.8 + 0.5 + 0.1} = \frac{(7.2, 2.4) + (3.0, 2.0) + (0.2, 0.9)}{1.4} = \frac{(10.4, 5.3)}{1.4} \approx (7.43, 3.79).$$

The simple (unweighted) average:

$$\bar{x} = \frac{(9,3) + (6,4) + (2,9)}{3} = \frac{(17,16)}{3} \approx (5.67, 5.33).$$



The soft centroid $(7.43, 3.79)$ is much closer to Harry's position than the simple average $(5.67, 5.33)$. This is because Harry has the highest Gryffindor posterior (weight 0.8), while Draco's weight (0.1) means his Slytherin-like traits barely influence the Gryffindor centroid. The simple average treats all three equally, so Draco's high cunning pulls the average upward and leftward, distorting the Gryffindor profile.

(b) K-means assignment is exactly nearest-centroid retrieval: the "query" is the student's trait vector x, and the "database" is the set of K centroids $\{\mu_1, \ldots, \mu_K\}$. The Sorting Hat retrieves the centroid with the smallest squared

distance to the student, just as a retrieval system finds the nearest item in a database. The only difference is that in clustering, the database (centroids) is learned and updated iteratively, whereas in standard retrieval the database is fixed.

Question 10.

1. K-means as a special case of MoG. K-means is equivalent to MoG in the limit $\sigma^2 \to 0$ (no observational noise). Under this assumption, the posterior $p(z = k|x_n)$ collapses to a one-hot vector, and the soft centroid update reduces to the hard average over assigned members. MoG generalizes k-means by introducing a noise parameter $\sigma^2 > 0$ that allows uncertainty in house assignment.

2. The σ^2 confidence dial. The parameter σ^2 gives the Sorting Hat a "confidence dial." When σ^2 is small, sorting is decisive — each student is assigned to one house with near-certainty. When σ^2 is larger, the Hat acknowledges ambiguity: "Hmm, difficult. You'd do well in Slytherin *or* Gryffindor." This is more honest than forcing a hard choice for students near the boundary between two houses. The Hat can express a degree of confidence in each assignment, which is valuable information for the student and the school.

3. More nuanced gradient. In k-means, each student fully contributes to exactly one house centroid and has zero influence on the others. This all-or-nothing update ignores the reality that boundary students (e.g., one who is both brave and cunning) carry information about multiple houses. The MoG gradient $\nabla_{\mu_k} = \frac{1}{\sigma^2} p(z = k|x_n)(x_n - \mu_k)$ uses the posterior $p(z = k|x_n)$ as a continuous weight, so a student who is 70% Gryffindor and 30% Slytherin contributes proportionally to both centroids. This produces smoother, more stable updates.

4. A downside of MoG. MoG is more computationally expensive: each update step requires computing the posterior over all K houses for each student (evaluating K Gaussian densities and normalizing), whereas k-means only requires finding the nearest centroid. MoG also introduces the hyperparameter σ^2, which must be tuned (or learned), adding complexity. For very large K or very clear-cut sorting scenarios, the added cost of soft assignment may not be justified.

Question 11.

(a) The quantization step is exactly the **k-means assignment step** from Q2. Each spatial latent vector $z_e^{(i)}$ plays the role of a data point (student), and the codebook entries $\{e_1, \ldots, e_K\}$ play the role of the centroids (house profiles). For each "student" $z_e^{(i)}$, VQ-VAE finds the nearest "centroid":

$$k_i^\star = \arg\min_k \|z_e^{(i)} - e_k\|^2,$$

which is identical to the k-means hard assignment. This happens independently at every spatial position in the grid, so one image produces many assignment decisions — one per grid cell.

Back to Contents

(b) If we collected all assigned vectors into a batch, the update would be the k-means centroid formula from Q3:

$$e_k \leftarrow \frac{1}{|\{i : k_i^\star = k\}|} \sum_{i:\, k_i^\star = k} z_e^{(i)}.$$

Each codebook entry e_k converges to the centroid (mean) of its assigned cluster in the latent space.

Important distinction: VQ-VAE does *not* run batch k-means. Because the encoder F changes every gradient step, the latent vectors $z_e^{(i)}$ are a moving target — so re-computing exact centroids over the full dataset is both impractical and meaningless. Instead, VQ-VAE updates codebook entries with an **exponential moving average** (EMA):

$$e_k \leftarrow \gamma\, e_k + (1-\gamma)\bar{z}_k,$$

where $\bar{z}_k$ is the mean of vectors assigned to e_k in the current minibatch and $\gamma \approx 0.99$. This is an *online* clustering algorithm: it tracks drifting clusters without ever reprocessing old data. The assignment step (nearest codebook entry) is the same as k-means, but the update step is adapted for a non-stationary setting. That said, some implementations do use batch k-means to *initialize* the codebook before training begins, giving the entries a reasonable starting spread across the latent space.

(c) Discretization converts each image into a sequence of codebook indices (integers from 1 to K). This is useful because:

- It turns image generation into a *sequence modeling* problem. After training VQ-VAE, DALL-E uses an autoregressive transformer (similar to GPT) to generate sequences of codebook indices, just as a language model generates sequences of word tokens. This lets DALL-E leverage the same powerful language modeling techniques for images.

- A discrete codebook of size K acts as a "visual vocabulary" — each codebook entry represents a recurring visual pattern (a patch of texture, a color gradient, an edge). Just as language has a finite vocabulary of words, VQ-VAE creates a finite vocabulary of visual elements. This compression makes the generation problem tractable: instead of generating continuous pixel values, the model only needs to choose from K options at each position.